PROTEGE+

Canal Nacional de Denúncias para Proteção do Cidadão

HackGov · Fase 6

Internet das Coisas · Inteligência Artificial · SQL Avançado · Estruturas de Dados

IoT | SQL Avançado | Estruturas de Dados

Gabriel Vasconcellos Gomes

RM 561601

Turma 2SIOA · Barueri, São Paulo · Entrega individual

Vídeo de demonstração (YouTube, não listado): **youtu.be/FHOxDr-g8I8**

Repositório: **github.com/Gabriell230G/ProtegeMais-Fase-6**

FIAP · 2026

Ficha técnica

Item	Conteúdo
Projeto	Protege+, Canal Nacional de Denúncias
Fase	6 de 6
Entrega	Documento, apresentação, vídeo e código-fonte
Aluno	Gabriel Vasconcellos Gomes, RM 561601
Turma	2SIOA
Município	Barueri, São Paulo
Vídeo, não listado	`youtu.be/FHOxDr-g8I8`
Repositório	github.com/Gabriell230G/ProtegeMais-Fase-6
Site publicado	gabriell230g.github.io/ProtegeMais-Fase-6
Back-end	Java 17, Spring Boot 3.3.4
Banco de demonstração	H2 em arquivo, perfil padrão
Banco de produção previsto	PostgreSQL; modelo físico em Oracle 19c
Testes automatizados	136, sem falhas
Classes Java de produção	79
Endpoints REST	38
Tabelas no modelo físico	15

Sobre a estrutura deste documento

Este documento foi organizado para ser conferido item a item contra o enunciado da Fase 6. Cada uma das quatro Partes abre com um quadro intitulado **O que o enunciado pede**, reproduzindo a exigência literal, e fecha com uma síntese que aponta onde cada exigência foi atendida.

Quatro princípios guiaram a redação, os mesmos adotados no documento da Fase 5 e mantidos aqui por terem se mostrado úteis.

Toda decisão de projeto é justificada, e a alternativa descartada é nomeada. Não basta dizer o que foi feito: o texto diz o que *não* foi feito e por quê. Uma escolha só é uma escolha quando existia outra opção viável, e o leitor precisa poder discordar com conhecimento de causa.

Toda afirmação técnica aponta para um arquivo. Quando o texto diz que uma regra existe, ele nomeia a classe, o método, a linha do script ou a tela em que ela está escrita. Uma afirmação que não pode ser conferida no repositório não deveria estar num documento de engenharia.

Todo número apresentado é reproduzível. As estatísticas das Partes 1 e 4 saem de uma base determinística, gerada pelo programa `database/gerar_carga_fase6.py` a partir da semente 561601. Reexecutar o gerador produz exatamente os mesmos 106 eventos e as mesmas 65 denúncias, em qualquer máquina.

Os limites do que foi construído são declarados. As seções 4.12, 5.11 e 6.2 listam o que não está resolvido, incluindo uma limitação conceitual da solução de agrupamento que só ficou visível depois de a tela estar pronta. Um documento que só relata acertos descreve uma demonstração, e não um sistema.

Sumário

Seção	Conteúdo	Pág.
	Introdução	**9**
1.1	O problema público que o sistema endereça	9
1.2	O que esta fase acrescenta	9
1.3	Como ler este documento	10
1.4	Método de trabalho e critério de verificação	11
	PARTE 1: Versão atual do projeto	**12**
2.1	Ponto de partida: o que a Fase 5 entregou	12
2.2	Evolução medida, número a número	12
2.3	Evolução 1: o barramento de eventos de dispositivos	13
2.4	Evolução 2: análise de relacionamento entre denúncias	14
2.5	Evolução 3: análise de dados com SQL avançado	15
2.6	Evolução 4: correções e melhorias no que já existia	15
2.7	O estado da suíte de testes	16
2.8	O achado que mudou o método: 116 testes e a aplicação não subia	17
2.9	Registro cronológico das evoluções	18
2.10	Síntese da Parte 1	18

Seção	Conteúdo	Pág.
	PARTE 2: Revisão e ajustes no backlog	**19**
3.1	Método de revisão	19
3.2	Estado das 35 histórias herdadas	19
3.3	Novas histórias: dispositivos de campo	22
3.4	Novas histórias: análise e eficiência dos dados	22
3.5	Novas histórias: qualidade e infraestrutura	23
3.6	Critérios de aceitação das histórias prioritárias	23
3.7	Histórias consideradas e recusadas	25
3.8	Definição de pronto	26
3.9	Quadro consolidado do backlog	27
	PARTE 3: Inteligência Artificial e IoT	**28**
4.1	O problema identificado	28
4.2	A escolha da tecnologia, e as alternativas descartadas	29
4.3	A Inteligência Artificial que já roda no projeto	31
4.4	Arquitetura: um barramento, três origens	33
4.5	Origem A: botão de pânico	34
4.6	Origem B: geofencing de medida protetiva	36
4.7	Origem C: totem público	37
4.8	Os dados utilizados	39

Seção	Conteúdo	Pág.
4.9	Integração com o sistema existente	40
4.10	Segurança e privacidade	41
4.11	Benefícios da solução	43
4.12	Limitações declaradas	44
4.13	O simulador de dispositivos	45
4.14	Síntese da Parte 3	45
	PARTE 4: Análise e eficiência dos dados	**47**
5.1	Método da análise com SQL	47
5.2	As seis consultas, seus resultados e o que eles informam	48
5.3	A carga determinística, e um erro de método que quase entrou	54
5.4	O problema das estruturas de dados: focos de reincidência	55
5.5	Por que não um GROUP BY	56
5.6	As quatro estruturas, e o que cada uma resolve	56
5.7	O detalhe que muda o custo	58
5.8	Análise de complexidade	59
5.9	O achado: foco não é o mesmo que volume	60
5.10	Por que a leitura de focos é auditada	62
5.11	Limitações declaradas da Parte 4	63
5.12	Verificação	63

Seção	Conteúdo	Pág.
5.13	Síntese da Parte 4	64
	Conclusão	**66**
6.1	O que foi entregue	66
6.2	O que não foi feito, e por quê	66
6.3	Próximos passos	67
6.4	Uma observação final sobre método	68
	Referências	**69**
	Legislação e normas do poder público brasileiro	69
	Algoritmos e estruturas de dados	69
	Engenharia de software, requisitos e testes	70
	Segurança da informação e privacidade	70
	Documentação das tecnologias utilizadas	71
	Nota sobre o uso das fontes	71
	Anexos	**73**
	Anexo A: rotas acrescentadas nesta fase	73
	Anexo B: objetos do banco acrescentados	74
	Anexo C: resultados completos das consultas	75
	Anexo D: defeitos encontrados nesta fase	75

Seção	Conteúdo	Pág.
	Anexo E: inventário de arquivos	78
	Anexo F: comandos de verificação	80
	Anexo G: roteiro do vídeo	81

Introdução

1.1 O problema público que o sistema endereça

O Protege+ é um canal nacional de denúncias construído para o programa HackGov, em parceria com a EGESP. Ele existe para receber relatos de violência, discriminação, abuso e assédio, classificá-los por urgência e entregá-los ao órgão competente com rastreabilidade.

O desenho do sistema parte de uma constatação que atravessa todas as seis fases: **o obstáculo principal de um canal de denúncias não é técnico, é de confiança.** Quem denuncia, muitas vezes, teme mais a exposição do que a impunidade. A Lei nº 13.608/2018, ao tratar do serviço telefônico de recebimento de denúncias, assegura ao denunciante o sigilo dos dados que possam identificá-lo (BRASIL, 2018b), e a Lei nº 13.460/2017, que disciplina a participação e a defesa do usuário de serviços públicos, trata o acesso ao canal como direito e não como favor (BRASIL, 2017); o Decreto nº 10.153/2019 detalha salvaguardas equivalentes no âmbito do Poder Executivo federal (BRASIL, 2019). As duas normas dizem a mesma coisa por caminhos diferentes: um canal que não protege quem fala deixa de receber a informação que justifica sua existência.

Essa constatação tem consequência de engenharia, e não apenas de política pública. Ela é a razão pela qual o anonimato, o mascaramento por perfil e a trilha de auditoria foram tratados nas fases anteriores como requisitos de primeira classe, e não como opções de configuração.

1.2 O que esta fase acrescenta

As fases anteriores construíram o canal: o portal público, o painel do órgão, a API em Java, o banco relacional, a trilha de auditoria encadeada e a análise estatística do tempo de resposta. Todas elas partem do mesmo pressuposto: **uma pessoa digita um relato.**

A Fase 6 questiona esse pressuposto. Existem situações em que a pessoa não pode digitar, não tem aparelho, ou não deveria precisar relatar coisa alguma porque o fato pode ser detectado sem ela. Três delas orientam o trabalho desta fase:

- quem está sob ameaça imediata precisa de um gesto, não de um formulário;
- quem tem medida protetiva não deveria ter que perceber e denunciar a aproximação do agressor: o descumprimento pode ser detectado;
- quem não tem celular, crédito ou privacidade em casa precisa de um ponto de atendimento físico que não peça identificação.

Essas três situações são endereçadas na Parte 3 por três origens de dispositivo que conversam com um mesmo barramento de eventos no servidor.

Em paralelo, a Parte 4 ataca um problema diferente e igualmente concreto: **a base cresceu, e a lista plana deixou de responder às perguntas da gestão.** Com mais de duas centenas de denúncias, saber

quantas chegaram importa menos do que saber *quais delas descrevem a mesma coisa*. Essa pergunta não se resolve com filtro nem com agrupamento de SQL, e a seção 5.5 explica por quê.

1.3 Como ler este documento

O quadro abaixo mapeia as exigências do enunciado às seções que as atendem. Ele é a chave de leitura para quem vai corrigir.

Enunciado da Fase 6	Onde está
Parte 1: relate as evoluções desde a última entrega	Capítulo 2
Parte 2: atualize o backlog com novas user stories	Seções 3.3 a 3.5
Parte 2: escreva critérios de aceitação das prioritárias	Seção 3.6
Parte 3: identifique um problema ou oportunidade	Seção 4.1
Parte 3: escolha a tecnologia mais adequada	Seção 4.2
Parte 3: justifique a escolha	Seções 4.2 e 4.3
Parte 3: explique os dados e a integração ao sistema	Seções 4.8 e 4.9
Parte 3: apresente benefícios e limitações	Seções 4.11 e 4.12
Parte 3: dê início à implementação	Seções 4.5 a 4.7
Parte 3: se já houver IA rodando, explique e justifique	Seção 4.3
Parte 4: consultas com agrupamentos, totais ou métricas	Seções 5.2.1, 5.2.2, 5.2.5 e 5.2.6
Parte 4: ao menos uma subconsulta ou função avançada	Seções 5.2.2, 5.2.3 e 5.2.4
Parte 4: apresente os resultados obtidos	Seção 5.2, e Anexo C
Parte 4: explique as informações que se extraem	Seção 5.2, em cada consulta
Parte 4: identifique uma situação de busca, ordenação ou relacionamento	Seção 5.4
Parte 4: árvores, busca, ordenação, grafos	Seções 5.6.1 a 5.6.4
Parte 4: análise de complexidade com notação Big O	Seção 5.8

Enunciado da Fase 6	Onde está
Parte 4: explique o problema e justifique a solução	Seções 5.4 e 5.5

1.4 Método de trabalho e critério de verificação

Esta fase foi desenvolvida com um método de verificação explícito, herdado da Fase 5 e endurecido depois de um achado descrito na seção 2.7: **nenhuma funcionalidade foi considerada pronta antes de ser executada.**

Na prática, isso significou três exigências cumulativas.

A primeira é que o código compile e os testes passem. A suíte saiu dos 81 testes publicados na Fase 5 para 136 ao final desta, todos executados a cada alteração pelo comando `build.cmd clean test`.

A segunda é que a aplicação suba. Essa exigência parece redundante diante da primeira, e a seção 2.7 mostra que não é: durante esta fase, o projeto esteve num estado em que **116 testes passavam e a aplicação não subia**. O defeito estava na costura entre o código e o framework, território que nenhum teste unitário visita.

A terceira é que a funcionalidade seja exercida pela interface, de ponta a ponta, com um ser humano clicando. Três defeitos desta fase, catalogados no Anexo D, só apareceram nesse último passo, e nenhum deles teria sido encontrado por leitura de código.

Esse método tem um custo: ele é lento, e obriga a refazer o caminho inteiro a cada alteração. O benefício está no Anexo D, que lista onze defeitos encontrados antes da entrega, e não depois dela.

PARTE 1: Versão atual do projeto

> **O que o enunciado pede**
>
> *Relate quais foram as evoluções no HackGov realizadas desde a última entrega de atividade.*

2.1 Ponto de partida: o que a Fase 5 entregou

A Fase 5 fechou com um sistema completo no eixo *pessoa digita, órgão atende*. Havia portal público, painel do órgão, API REST em Java com Spring Boot, persistência relacional, autenticação com JWT, controle de acesso por perfil, mascaramento de dados pessoais, trilha de auditoria encadeada por hash, upload auditado de evidências e um relatório estatístico com medidas de posição e dispersão sobre o tempo de resposta.

Aquele documento também declarou, na seção 6.3, três frentes em aberto. Duas delas são atacadas nesta fase: **a ausência de qualquer canal que não dependa de o cidadão digitar** e **a incapacidade de o painel relacionar denúncias entre si**. A terceira, a federação com sistemas de órgãos parceiros, permanece fora de escopo, e a seção 6.2 deste documento explica por quê.

2.2 Evolução medida, número a número

A tabela abaixo compara o estado do projeto no fechamento da Fase 5 com o estado no fechamento desta fase. Todos os valores foram apurados no repositório na data desta versão, pelos comandos registrados no Anexo F.

Indicador	Fase 5	Fase 6	Variação
Classes Java de produção	63	79	+16
Controladores REST	9	11	+2
Endpoints REST	31	38	+7
Testes automatizados	81	136	+55
Tabelas no modelo físico	12	15	+3
Restrições CHECK	30	45	+15
Restrições UNIQUE	6	8	+2
Chaves estrangeiras	11	10	recontagem
Índices	13	15	+2

Indicador	Fase 5	Fase 6	Variação
Módulos JavaScript	27	30	+3
Folhas de estilo	7	10	+3
Telas no painel do órgão	8	10	+2
Scripts SQL versionados	3	5	+2
Chaves de tradução por idioma	363	373	+10

A coluna **Fase 5** reproduz os valores publicados na seção 1.2 do documento daquela fase, e não uma medição feita agora sobre uma cópia de trabalho. Essa distinção importa: entre as duas entregas houve a Atividade 4 do Enterprise Challenge, que acrescentou o modo de demonstração offline (`js/demo-offline.js`) sem alterar o back-end. Parte da variação de módulos JavaScript vem de lá, e atribuí-la a esta fase seria inflar o resultado. A coluna **Fase 6** foi medida no repositório pelos comandos do Anexo F.

Três observações, porque números de volume são fáceis de apresentar e fáceis de superinterpretar.

A primeira é que **volume não é qualidade**. Dezesseis classes a mais não provam nada sozinhas. O que sustenta esta fase não é o crescimento de 25% nas classes de produção, é o crescimento de 68% na suíte de testes, proporção deliberada e discutida na seção 2.6.

A segunda é que **a variação de +3 folhas de estilo esconde uma operação de redução**, não de acréscimo. Uma das três, `css/tema-escuro.css`, não é código novo: é um bloco que vivia dentro de uma tag `<style>` no `index.html` e foi extraído para poder ser reaproveitado por uma segunda página. A seção 2.6.3 descreve o caso.

A terceira é que **as chaves estrangeiras não mudaram no código, mas a contagem mudou**. O resumo no fim do arquivo `01_ddl_oracle.sql` declarava onze desde a Fase 5, e a apuração desta fase, contando as cláusulas `FOREIGN KEY` efetivamente declaradas, encontrou dez. Nenhuma restrição foi removida: o número publicado é que estava um acima, e tanto a tabela acima quanto o arquivo passam a exibir o valor apurado. As três tabelas novas, por sua vez, não referenciam `DENUNCIA` por chave estrangeira: elas guardam o protocolo como texto. A justificativa está na seção 4.4.3, e ela é uma das decisões mais discutíveis desta fase.

2.3 Evolução 1: o barramento de eventos de dispositivos

A mudança estruturalmente mais significativa desta fase é a entrada de uma segunda classe de emissor no sistema. Até a Fase 5, toda denúncia nascia de um formulário preenchido por uma pessoa. A partir desta fase, existe um caminho paralelo: **um aparelho relata um evento, e o servidor decide se aquilo vira um caso.**

A implementação está concentrada em dez arquivos novos, listados abaixo com o respectivo tamanho, e detalhada no Capítulo 4.

Arquivo	Linhas	Papel
`model/EventoIot.java`	113	Entidade do barramento, com discriminador de origem
`model/ZonaExclusao.java`	89	Perímetro de medida protetiva
`model/PontoTotem.java`	82	Cadastro de totens instalados
`repository/EventoIotRepository.java`	26	Consultas do barramento
`repository/ZonaExclusaoRepository.java`	14	Busca do perímetro ativo
`repository/PontoTotemRepository.java`	20	Busca do totem habilitado
`service/EventoIotService.java`	522	Regras das três origens
`controller/IotController.java`	145	Três rotas de dispositivo e duas de leitura
`dto/` (seis arquivos)	214	Contratos de entrada e saída
`database/04_ddl_fase6.sql`	163	Modelo físico das três tabelas

A decisão central, discutida em 4.4, foi **não criar três subsistemas**. As três origens escrevem na mesma tabela, passam pelo mesmo controlador e aparecem na mesma tela, distinguidas por uma coluna discriminadora.

2.4 Evolução 2: análise de relacionamento entre denúncias

A segunda evolução ataca uma limitação que a Fase 5 declarou e não resolveu: o painel sabia contar denúncias e não sabia relacioná-las.

Foram criados `service/ReincidenciaService.java`, com 397 linhas, e `controller/ReincidenciaController.java`, com 54, mais a tela **Focos de reincidência** no painel do órgão, composta por `js/reincidencia.js` e `css/reincidencia.css`.

O serviço aplica quatro estruturas de dados a um único problema, descrito em detalhe no Capítulo 5: árvore balanceada para o recorte temporal, ordenação para agrupamento, busca binária para localização e grafo com busca em largura para a detecção de componentes conexos.

2.5 Evolução 3: análise de dados com SQL avançado

O repositório ganhou dois scripts versionados e um programa gerador.

`database/05_consultas_fase6.sql` reúne seis consultas analíticas sobre o barramento de dispositivos, usando agrupamento com `ROLLUP` e `GROUPING`, funções de janela (`RANK`, `SUM OVER`, `LAG`), subconsulta correlacionada no `HAVING` e expressão de tabela comum. `database/05_resultados_fase6.txt` registra a saída de cada uma e a leitura do que ela informa.

`database/gerar_carga_fase6.py` é o programa que produz a carga de demonstração. Ele está no repositório por uma razão de método: a carga da Parte 4 não foi digitada nem colada, é saída de um programa com semente fixa. Quem corrige pode executá-lo e obter exatamente os mesmos 106 eventos e as mesmas 65 denúncias, e com isso conferir na origem qualquer número citado neste documento (PENG, 2011).

2.6 Evolução 4: correções e melhorias no que já existia

Nem toda evolução desta fase é funcionalidade nova. Quatro intervenções corrigiram problemas em código entregue em fases anteriores.

2.6.1 Mapeamento inválido na entidade do barramento

`EventoIot` declarava `@Column(precision = 9, scale = 6)` sobre campos `Double`. Escala é um conceito de tipo decimal exato, e o Hibernate recusa aplicá-la a um tipo de ponto flutuante (RED HAT, [s.d.]). O caso é tratado em detalhe na seção 2.7 porque ele revelou um problema maior que ele mesmo.

2.6.2 Cabeçalho personalizado ausente na política de CORS

O controlador de dispositivos autentica pelo cabeçalho `X-Device-Key`. Aceitar o cabeçalho no controlador não basta: um cabeçalho personalizado torna a requisição do navegador *não simples*, e o navegador envia antes uma requisição `OPTIONS` de verificação prévia (MOZILLA, [s.d.]). A lista de cabeçalhos aceitos é declarada no `CorsConfigurationSource` do Spring (VMWARE, [s.d.]). Sem `X-Device-Key` na lista de cabeçalhos permitidos de `SecurityConfig`, essa verificação era recusada e o `POST` sequer chegava a existir. O sintoma no navegador é um erro de CORS que parece falha de rede e não aponta para o cabeçalho.

2.6.3 Tema escuro duplicado entre páginas

O bloco de variáveis do modo escuro vivia dentro de uma tag `<style>` no `index.html`. Quando o simulador de dispositivos passou a precisar do mesmo tema, havia duas saídas: copiar as cores ou extraí-las. Manter duas cópias das mesmas cores em dois arquivos garante que uma delas fique para trás, e a que fica para trás é sempre a que ninguém está olhando. O bloco foi extraído para `css/tema-escuro.css` e as duas páginas passaram a carregá-lo.

Junto da extração, uma auditoria varreu todas as folhas de estilo procurando regra com fundo claro fixo sem par escuro. Foram encontrados 29 seletores em três categorias distintas, e cada categoria exigiu um tratamento próprio: cor fixa em CSS, que ganhou regra escura correspondente; estilo *inline* no HTML, que precisou virar classe, porque estilo *inline* vence qualquer folha; e cor decidida em JavaScript, o caso dos gráficos, porque um elemento `canvas` não herda CSS e a cor da grade era pintada por código.

2.6.4 Assistente virtual sobreposto à mesa de trabalho

O balão do VigIA aparecia flutuando sobre o painel do órgão. Ele atende *quem denuncia*: explica o formulário, tira dúvida sobre anonimato e ensina a acompanhar o protocolo. Nada disso serve a quem está do outro lado do balcão, e um balão flutuante sobre uma tabela de trabalho é um obstáculo. O componente passou a ser ocultado no painel, seguindo a mesma regra já aplicada ao botão de pânico. O VigIA permanece no painel onde de fato auxilia o servidor: no bloco de análise do Resumo.

2.7 O estado da suíte de testes

A suíte saiu dos 81 testes publicados na Fase 5 para 136. A distribuição por classe está abaixo, com a indicação do que é novo nesta fase.

Classe de teste	Testes	Situação
`ContextoSobeTest`	1	Nova nesta fase
`EventoIotServiceTest`	35	Nova nesta fase
`ReincidenciaServiceTest`	19	Nova nesta fase
`EstatisticaServiceTest`	14	Herdada
`RelatorioServiceTest`	11	Herdada
`EvidenciaServiceTest`	9	Herdada
`ExportacaoServiceTest`	8	Herdada
`AssinaturaArquivoTest`	8	Herdada
`AuditoriaServiceTest`	7	Herdada
`NomeArquivoUtilTest`	7	Herdada
`FilaAtendimentoServiceTest`	6	Herdada
`PilhaAcoesServiceTest`	5	Herdada
`ScoreServiceTest`	3	Herdada
`VigiaServiceTest`	3	Herdada

Classe de teste	Testes	Situação
Total	**136**	

A proporção merece comentário. As classes de produção cresceram 25% e a suíte cresceu 68%. Isso não é acidente: as duas funcionalidades desta fase são algoritmos com regras de fronteira, e regra de fronteira é exatamente o que se erra ao escrever e o que ninguém percebe ao ler. Dos 55 testes novos, 21 verificam bordas: o instante exato em que a janela de repique expira, a distância exata do raio do perímetro, o intervalo exato do elo entre duas denúncias, o limiar exato de saturação de um agrupamento.

2.8 O achado que mudou o método: 116 testes e a aplicação não subia

Este é o episódio mais instrutivo da fase, e por isso ele ganha seção própria em vez de uma linha no anexo de defeitos.

Em determinado momento desta fase, o projeto estava no seguinte estado: **116 testes passando, `BUILD SUCCESS`, e a aplicação não subia.** A tentativa de executá-la terminava em `BeanCreationException` na criação do `entityManagerFactory`, com a mensagem *scale has no meaning for SQL floating point types*.

A causa era a anotação inválida descrita em 2.6.1. O que interessa aqui não é a causa, é o motivo de nenhum dos 116 testes ter percebido: **todos eram unitários, e nenhum carregava o contexto do Spring.** Uma suíte assim testa o que o programador escreveu e ignora o que o framework exige. A costura entre os dois ficava sem cobertura.

O agravante é que esse era o primeiro erro que um avaliador encontraria, porque a primeira coisa que alguém faz ao receber um projeto é executá-lo, não rodar a suíte.

A correção foi dupla. O mapeamento foi ajustado, com a precisão de seis casas declarada onde ela de fato vale, no `NUMBER(9,6)` do modelo físico. E foi criada a classe `ContextoSobeTest`, um teste de fumaça sem nenhuma asserção, que sobe o contexto inteiro em banco de memória: mapeamento das entidades, criação do esquema, execução do `data.sql` e montagem de todos os *beans*. A falha dele é a exceção lançada na montagem, e a mensagem dela já é o diagnóstico.

Ele roda em `jdbc:h2:mem` e não no arquivo da demonstração, para que a suíte nunca altere os dados que a apresentação vai usar.

A correção seguiu a disciplina de escrever primeiro o teste que falha e só então o conserto (BECK, 2002), e o teste criado corresponde ao topo da pirâmide descrita por Fowler ([s.d.]): poucos testes de integração, caros e lentos, sobre uma base larga de testes unitários. O projeto tinha a base e não tinha o topo. A lição, registrada aqui porque vale além deste trabalho: **um teste que nunca falhou não provou nada até o dia em que falha.**

2.9 Registro cronológico das evoluções

#	Evolução	Verificação
1	Entidade, repositório e DDL do barramento de eventos	Build com 93 testes
2	Origem A: botão de pânico, com regra de repique	12 testes novos
3	Origem B: geofencing, com Haversine e cooldown	11 testes novos, 104 no total
4	Origem C: totem público, com janela de repique própria	12 testes novos, 116 no total
5	Simulador de dispositivos e tela do painel	Execução ponta a ponta
6	Correção do mapeamento e teste de fumaça	117 testes, aplicação sobe
7	Auditoria completa do tema escuro	Inspeção das 10 folhas
8	Carga determinística e seis consultas SQL	Resultados no Anexo C
9	Focos de reincidência: quatro estruturas	15 testes novos, 132 no total
10	Distinção entre foco e fluxo contínuo	4 testes novos, 136 no total

2.10 Síntese da Parte 1

O sistema deixou de pressupor que toda denúncia nasce de alguém digitando, e passou a saber relacionar denúncias entre si. Essas são as duas evoluções funcionais da fase.

Além delas, a fase corrigiu quatro problemas em código anterior e fechou uma lacuna estrutural na estratégia de testes, que existia desde a Fase 5 e não havia sido percebida porque a suíte, sozinha, não era capaz de percebê-la.

PARTE 2: Revisão e ajustes no backlog

> **O que o enunciado pede**
>
> *Reavalie o Product Backlog criado nas fases anteriores e: atualize-o com novas user stories relacionadas à integração backend, frontend e banco de dados; escreva os critérios de aceitação das user stories prioritárias.*

3.1 Método de revisão

O backlog chegou a esta fase com 35 histórias, numeradas de US01 a US35. A revisão seguiu o mesmo critério adotado na Fase 5, que se mostrou produtivo porque devolveu três histórias ao backlog naquela ocasião: **uma história só permanece marcada como concluída se for possível apontar o arquivo, a rota ou a tela que a implementa, e executá-la.**

Duas regras complementares foram acrescentadas nesta fase, ambas nascidas de episódios concretos descritos no Capítulo 2.

A primeira: **uma história cuja implementação não sobe não está concluída**, ainda que seus testes unitários passem. A regra nasceu do estado descrito na seção 2.7, em que 116 testes passavam e a aplicação não subia, e ela é a razão de existir da US44.

A segunda: **uma história de interface não está concluída enquanto não for exercida por uma pessoa clicando**. Três defeitos desta fase, catalogados no Anexo D, passaram por revisão de código sem serem notados, e apareceram no primeiro uso real da tela.

A escrita das histórias segue o formato consagrado por Cohn (2004): papel, desejo e motivo, com o motivo carregando o valor que justifica a existência da história. Os critérios de aceitação seguem a orientação de Wiegers e Beatty (2013) de que um critério só é útil se for verificável de forma binária por alguém que não escreveu o código.

3.2 Estado das 35 histórias herdadas

A tabela abaixo registra o estado de cada história herdada ao final desta fase. Ela é reproduzida aqui para que este documento não dependa da leitura do PDF anterior, conforme o princípio de autossuficiência declarado na abertura.

#	História	Estado ao fim da Fase 6
US01	Cidadão registra denúncia	Concluída
US02	Denúncia anônima	Concluída
US03	Informar local do incidente	Concluída

#	História	Estado ao fim da Fase 6
US04	Servidor visualiza painel	Concluída
US05	Data e hora das denúncias	Concluída
US06	Preenchimento por GPS	Concluída
US07	Relato em áudio	Concluída
US08	Anexar fotos e documentos	Concluída
US09	QR Code do protocolo	Concluída
US10	Compartilhar link do protocolo	Concluída
US11	Filtros avançados no backlog	Concluída
US12	Histórico de status rastreável	Concluída
US13	Mapa de distribuição geográfica	Concluída
US14	Acesso em inglês	Concluída
US15	Modo seguro (*fake page*)	Concluída
US16	Tema claro e escuro	**Reaberta e concluída nesta fase**
US17	Componente React do formulário	Concluída
US18	Validação e sanitização de entradas	Concluída
US19	Trilha de auditoria filtrável	Concluída
US20	Fila de priorização automática	Concluída
US21	Desfazer última movimentação do Kanban	Concluída
US22	Indicador de origem dos dados	Concluída
US23	Painel estatístico com dispersão	Concluída
US24	Perfis com permissões distintas	Concluída, estendida nesta fase
US25	Registro imutável e encadeado	Concluída, estendida nesta fase
US26	Mascaramento por perfil	Concluída

#	História	Estado ao fim da Fase 6
US27	Limite de tentativas por IP	Concluída
US28	Anexar evidências pela API	Concluída
US29	Download auditado de evidência	Concluída
US30	Exportação em CSV auditada	Concluída
US31	Portal e painel servidos pela API	Concluída
US32	Ficha completa do caso no painel	Concluída
US33	Tentativas recusadas na trilha	Concluída
US34	Recusa de arquivo por assinatura	Concluída
US35	Remoção de anexo com motivo	Concluída

3.2.1 A reabertura da US16

Uma única história herdada voltou ao backlog nesta fase, e o motivo merece registro porque ele é um caso de critério de aceitação mal escrito, e não de código mal feito.

A US16 dizia: *como usuário, quero alternar entre tema claro e escuro, para reduzir o cansaço visual em ambientes de pouca luz*. O critério de aceitação original era *o botão alterna o tema e a escolha persiste entre sessões*. Esse critério era verificável, era binário, e estava sendo cumprido. O problema é que ele descreve o **mecanismo** e não o **resultado**.

A auditoria descrita na seção 2.6.3 encontrou 29 seletores com fundo claro fixo sem par escuro, concentrados nas telas do painel construídas na Fase 5. O mecanismo funcionava; o tema não existia naquelas telas. Um usuário que alternasse o tema com o relatório estatístico aberto veria a maior parte da tela continuar branca.

A história foi reaberta e o critério reescrito para descrever resultado:

> Em qualquer tela do sistema, com o tema escuro ativo, nenhum elemento de fundo permanece claro, e todo texto mantém contraste legível. A verificação percorre as dez telas do painel, a página pública e o simulador.

A lição é registrada porque vale além deste caso: **um critério que descreve o mecanismo é cumprido por um mecanismo que não produz o resultado.**

3.2.2 Extensões sem reabertura

Duas histórias foram estendidas sem voltar ao backlog, porque a implementação original continuou válida e apenas ganhou alcance.

A **US24**, sobre perfis e permissões, passou a cobrir as duas telas novas. O perfil Auditor não recebeu acesso a **Dispositivos de campo** nem a **Focos de reincidência**: ele fiscaliza o uso do sistema e não opera o atendimento. A decisão está em `js/acesso.js` e é discutida na seção 4.10.4.

A **US25**, sobre auditoria, passou a cobrir as leituras das duas telas novas. Ambas as rotas de consulta são anotadas com `@Auditavel(acao = CONSULTA_SENSIVEL)`. A justificativa para tratar a leitura de focos como sensível está na seção 5.10.

3.3 Novas histórias: dispositivos de campo

As histórias US36 a US40 nascem do problema descrito na seção 4.1.

#	História
US36	Como pessoa em situação de risco imediato, quero acionar um pedido de socorro com um único gesto físico, sem precisar desbloquear aparelho nem escrever, para que o pedido chegue quando eu não tiver condição de digitar.
US37	Como pessoa protegida por medida judicial, quero que a aproximação do monitorado seja detectada pelo sistema, para que eu não dependa de perceber e denunciar o descumprimento.
US38	Como cidadão sem telefone, crédito ou privacidade em casa, quero registrar uma denúncia em um equipamento público, sem me identificar, para que a falta de aparelho não me deixe fora do canal.
US39	Como gestor do órgão, quero ver o que os dispositivos enviaram e o que o servidor decidiu sobre cada envio, para poder confiar no alerta e para detectar equipamento com defeito.
US40	Como avaliador ou desenvolvedor, quero exercitar as três origens de dispositivo sem possuir o hardware, para verificar o comportamento do servidor de ponta a ponta.

3.4 Novas histórias: análise e eficiência dos dados

As histórias US41 a US43 nascem do problema descrito na seção 5.4.

#	História
US41	Como gestor do órgão, quero consultas que respondam perguntas de operação sobre o parque de dispositivos, para decidir manutenção e instalação com base em dado e não em impressão.
US42	Como gestor do órgão, quero saber quais denúncias descrevem a mesma situação em curso, mesmo sendo anônimas e separadas, para tratar um foco como um caso e não como vários.
US43	Como gestor do órgão, quero distinguir um surto que começa e termina da demanda constante de um lugar, para não montar uma operação pontual contra uma rotina.

3.5 Novas histórias: qualidade e infraestrutura

#	História
US44	Como desenvolvedor, quero que a suíte de testes falhe quando a aplicação não conseguir subir, para que um defeito de mapeamento não chegue ao avaliador travestido de *build* verde.
US45	Como usuário do painel, quero que o tema escuro valha em todas as telas, para que alternar o tema não deixe metade da interface acesa.

A US45 é a reescrita da US16 reaberta, e aparece nas duas listas de propósito: na tabela de herdadas como reabertura, e aqui como história nova, porque o critério de aceitação mudou.

3.6 Critérios de aceitação das histórias prioritárias

A priorização seguiu dois eixos: **risco à pessoa** e **exigência do enunciado**. As histórias US36, US37 e US38 concentram o primeiro; a US42 concentra o segundo. As cinco histórias abaixo são as prioritárias, e apenas elas recebem critérios detalhados, conforme pede o enunciado.

3.6.1 US36: acionamento por botão de pânico

1. Uma requisição `POST /api/iot/acionamento` com chave de dispositivo válida retorna `201` e o protocolo do caso aberto.
2. Uma requisição sem o cabeçalho `X-Device-Key`, ou com chave incorreta, retorna `401` e **nenhum evento é gravado**.

3. O caso é aberto pelo servidor. O corpo da requisição não contém, e não pode conter, classificação de urgência.
4. Um segundo acionamento do mesmo aparelho dentro de 120 segundos é gravado com situação `DUPLICADO`, vinculado ao protocolo do primeiro, **sem abrir caso novo**.
5. Um acionamento do mesmo aparelho após 120 segundos abre caso novo.
6. A coordenada gravada usa ponto decimal, independentemente da localidade da máquina que executa a aplicação.
7. O evento é registrado na trilha de auditoria.

O critério 4 é o que distingue uma pessoa com medo de três ocorrências. O critério 6 nasceu de um defeito real, descrito no Anexo D, item 1.

3.6.2 US37: detecção de descumprimento de medida protetiva

1. Uma requisição `POST /api/iot/telemetria` com chave válida retorna `202`.
2. Quando a posição informada está fora do perímetro, o evento é gravado com situação `ACEITO` e detalhe `Fora do perimetro`, e **nenhum caso é aberto**.
3. Quando a posição está dentro do perímetro, o servidor abre um caso sozinho, sem intervenção humana.
4. **A resposta ao aparelho não contém a distância nem o raio.** Este critério é de segurança, e a justificativa está na seção 4.6.2.
5. Leituras subsequentes dentro do perímetro, em menos de 600 segundos do último alerta, são gravadas como `DUPLICADO` e não abrem caso novo.
6. A distância é calculada pela fórmula de Haversine, e não por distância euclidiana sobre as coordenadas.
7. Um equipamento sem medida protetiva ativa tem a leitura descartada com registro, e não silenciosamente ignorada.
8. A borda não viola: uma posição exatamente sobre o raio está fora do perímetro.

3.6.3 US38: registro em totem público

1. Uma requisição `POST /api/iot/totem` com chave válida e relato suficiente retorna `201` com protocolo.
2. **O corpo da requisição não contém coordenada.** O lugar do registro vem do cadastro da instalação, e a justificativa está na seção 4.7.1.
3. Um totem desconhecido, ou cadastrado como inativo, recebe `422` e o evento é gravado como `DESCARTADO`.
4. Um relato com menos de 15 caracteres, já descontados espaços e caracteres de controle, recebe `422` e **o texto não é gravado**.
5. Um relato acima do limite de gravação é **cortado, não recusado**.
6. O registro é sempre anônimo. Não há campo de identificação na requisição.

7. Um reenvio do mesmo aparelho dentro de 20 segundos é tratado como duplicata. Um envio após 20 segundos é caso novo, ainda que do mesmo aparelho.
8. A resposta informa por quantos segundos o protocolo pode permanecer na tela.

O critério 7 é o que diferencia esta história da US36, e a diferença é deliberada: a janela do totem é seis vezes menor que a do botão de pânico. A justificativa está na seção 4.7.2.

3.6.4 US39: tela de dispositivos de campo

1. A tela exibe o total de eventos por origem e a lista dos mais recentes.
2. Cada linha mostra **o que o servidor decidiu**, e não apenas o que o aparelho enviou.
3. As três situações são visualmente distintas: aceito, duplicado e descartado.
4. A tela se atualiza sozinha enquanto está aberta, e **para de consultar a API quando o usuário sai dela**.
5. **Sem conexão com a API, a tela informa a ausência de dados e não exibe exemplos.** A justificativa está na seção 4.9.3.
6. Os perfis Administrador, Gestor e Atendente acessam a tela; o perfil Auditor não.

3.6.5 US42: focos de reincidência

1. A tela agrupa denúncias encadeadas no tempo e exibe cada agrupamento com município, tipo, quantidade, período, duração e gravidade.
2. Duas denúncias são ligadas quando são do mesmo tipo, do mesmo município e estão a no máximo 14 dias uma da outra.
3. **O agrupamento é transitivo**: se A liga em B e B liga em C, os três formam um único foco, ainda que A e C estejam a mais de 14 dias de distância.
4. Quando a cadeia se interrompe por mais de 14 dias, o foco termina e um novo pode começar.
5. Um agrupamento só é apresentado a partir de três denúncias.
6. A janela de análise é selecionável pelo usuário entre 30, 90, 180 e 365 dias.
7. O parâmetro que governa o agrupamento é exibido na tela, e não apenas aplicado.
8. A consulta é auditada como leitura sensível.

3.7 Histórias consideradas e recusadas

Esta seção existe porque um backlog que só registra o que foi aceito não mostra critério nenhum. As cinco propostas abaixo foram levantadas durante o planejamento desta fase e recusadas, com o motivo registrado.

Proposta	Motivo da recusa
Aplicativo para a pessoa protegida,	Transfere para a vítima o papel de operadora do sistema e a

Proposta	Motivo da recusa
com alerta de aproximação	mantém em estado de vigilância permanente. O alerta deve chegar a quem tem o dever de agir, não a quem está em risco.
Geocodificação reversa das coordenadas por serviço externo	Enviaria a coordenada de uma pessoa em risco a um terceiro, fora do controle do órgão. O sistema grava a coordenada como texto aproximado e não a traduz.
Agrupamento de denúncias por similaridade de texto	Exigiria modelo de linguagem sobre o relato, que é o dado mais sensível da base. O ganho não compensa o risco de o texto sair do servidor, e a alternativa adotada usa apenas metadados.
Identificação de reincidência **por pessoa**	Contradiz a garantia de anonimato. Reidentificar autores de relatos anônimos por cruzamento de atributos é exatamente o risco descrito por Rocher, Hendrickx e De Montjoye (2019), e o sistema não deve construir a ferramenta que ele existe para evitar.
Notificação automática à autoridade policial na violação de perímetro	Depende de integração institucional que não existe e cuja ausência não pode ser simulada com honestidade. Declarada como próximo passo na seção 6.3.

A terceira e a quarta recusas merecem uma observação conjunta. As duas propostas eram tecnicamente viáveis e teriam produzido um resultado mais impressionante na demonstração. Foram recusadas porque **o valor de um canal de denúncias está na confiança que ele sustenta**, e uma funcionalidade que enfraquece o anonimato destrói mais valor do que qualquer agrupamento adiciona.

3.8 Definição de pronto

A definição de pronto desta fase acrescenta dois itens à da Fase 5.

1. A história tem critérios de aceitação verificáveis de forma binária.
2. Existe teste automatizado cobrindo as regras de fronteira.
3. O comando `build.cmd clean test` termina sem falhas.
4. **A aplicação sobe.** Verificado pelo `ContextoSobeTest` e pela execução.
5. **A funcionalidade foi exercida pela interface por uma pessoa.**
6. A decisão de projeto está registrada em comentário no próprio código, e não apenas neste documento.

O item 6 pode parecer excessivo, e é deliberado: um documento se perde, e o código não. As decisões desta fase estão escritas nos arquivos que as implementam, e este documento as reúne e as justifica com fontes.

3.9 Quadro consolidado do backlog

Faixa	Quantidade	Situação
US01 a US35, herdadas	35	Concluídas, sendo 1 reaberta e reconcluída
US36 a US40, dispositivos de campo	5	Concluídas nesta fase
US41 a US43, análise de dados	3	Concluídas nesta fase
US44 e US45, qualidade	2	Concluídas nesta fase
Total	**45**	

PARTE 3: Inteligência Artificial e IoT

> **O que o enunciado pede**
>
> *Analise e identifique ao menos uma evolução que possa ser realizada no projeto com Inteligência Artificial ou IoT. O grupo deverá: identificar um problema ou oportunidade de melhoria; escolher a tecnologia mais adequada; justificar a escolha; explicar os dados utilizados e como a nova funcionalidade se integra ao sistema; apresentar benefícios e limitações da solução; dar início à implementação desta nova funcionalidade. Caso exista no projeto, já implementada e rodando, uma funcionalidade de IA ou IoT, explicar e justificar.*

4.1 O problema identificado

Todo o sistema construído até a Fase 5 repousa sobre um pressuposto que nunca havia sido examinado: **alguém abre o canal, lê o formulário, escolhe o tipo, escreve o relato e envia.**

Esse pressuposto exclui, por construção, três situações em que o canal é mais necessário.

4.1.1 Quem não pode digitar

O ato de denunciar exige tempo, privacidade e capacidade de escrever. Em risco imediato, nenhuma das três condições existe. A pessoa que está sendo ameaçada naquele instante não desbloqueia o telefone, não escolhe entre cinco tipos de denúncia e não redige um parágrafo.

A consequência é conhecida: **o canal recebe o relato depois, quando a urgência já passou, ou não recebe.** O sistema classifica bem o que recebe, e o que ele não recebe é justamente o mais grave.

O que resolve essa situação não é uma tela melhor. É um gesto: um botão físico que não precisa ser lido, escolhido nem preenchido.

4.1.2 Quem não deveria precisar denunciar

A Lei nº 11.340/2006 prevê medidas protetivas de urgência, entre elas o afastamento do agressor e a proibição de aproximação da ofendida, fixando limite mínimo de distância (BRASIL, 2006). A monitoração eletrônica de pessoas foi introduzida no ordenamento pela Lei nº 12.258/2010 e regulamentada pelo Decreto nº 7.627/2011 (BRASIL, 2010; BRASIL, 2011).

Existe, portanto, uma medida judicial que fixa uma distância e existe um equipamento capaz de informar posição. Mesmo assim, no desenho vigente até a Fase 5, **o descumprimento só chegaria ao canal se a pessoa protegida percebesse a aproximação e denunciasse**.

Isso é um problema de desenho, e não de tecnologia. Ele coloca sobre quem está em risco a tarefa de vigiar o próprio agressor. A informação necessária para detectar o descumprimento já existe em dois lugares, a decisão judicial e a posição do equipamento, e falta apenas alguém cruzá-las.

4.1.3 Quem não tem aparelho

O canal é digital e pressupõe telefone, crédito de dados e, sobretudo, privacidade para usá-lo. A terceira condição é a que o desenho costuma esquecer: em situação de violência doméstica, o aparelho pode estar sob vigilância de quem convive com a vítima.

O Protege+ já endereçava parte disso com o **modo seguro**, que disfarça a tela, entregue na US15. Mas o modo seguro protege quem já está no canal. Ele não ajuda quem não tem por onde entrar.

Um ponto de atendimento físico, instalado onde a pessoa já vai por outro motivo, resolve a falta de aparelho e a falta de privacidade ao mesmo tempo: uma unidade básica de saúde, um terminal rodoviário ou um centro de referência de assistência social são lugares onde estar não levanta suspeita.

4.2 A escolha da tecnologia, e as alternativas descartadas

O enunciado permite escolher entre Inteligência Artificial e IoT. A escolha desta fase foi **IoT**, e a justificativa depende de comparar o que cada tecnologia resolve nos três problemas acima.

4.2.1 Por que não mais Inteligência Artificial

A tentação natural era ampliar o VigIA, que já roda no projeto, com modelos mais sofisticados: classificação de gravidade por aprendizado supervisionado, extração de entidades do relato, detecção de risco por análise de linguagem.

Essa via foi examinada e descartada por três razões.

A primeira é que **ela não toca nenhum dos três problemas**. Um classificador melhor melhora o tratamento do relato que chegou; nenhum dos três problemas é de tratamento, todos são de chegada. Investir onde o sistema já é forte, ignorando onde ele não existe, é confortável e improdutivo.

A segunda é de risco. Um modelo estatístico treinado sobre relatos aprende as correlações da base, e uma base de denúncias carrega vieses de quem consegue denunciar. O resultado previsível é o modelo priorizar melhor os perfis que já são bem atendidos. A própria seção 5.9 deste documento mostra um caso menor da mesma família, em que uma medida aparentemente neutra descrevia o volume de um lugar em vez de destacar algo nele.

A terceira é de explicabilidade. O VigIA atual declara a origem de cada análise e opera por regras legíveis quando não há chave de API configurada. Uma decisão que afeta a ordem de atendimento de uma denúncia de violência precisa poder ser explicada a quem é afetado por ela, e um modelo estatístico entrega correlação, não explicação.

4.2.2 Por que IoT

IoT resolve os três problemas pela mesma via: **deslocar a origem do relato da tela para o mundo físico.**

Problema	O que IoT entrega
Quem não pode digitar	Um gesto substitui o formulário
Quem não deveria precisar denunciar	O equipamento informa posição, o servidor detecta
Quem não tem aparelho	O equipamento está no lugar, não no bolso

Há ainda uma razão de aderência ao domínio: **o Estado brasileiro já opera monitoração eletrônica.** A tecnologia não precisa ser inventada nem contratada de zero; o que falta é o canal de denúncias conversar com ela.

4.2.3 As alternativas dentro de IoT

Escolhido o caminho, três desenhos foram considerados.

Desenho	Avaliação
Um subsistema por tipo de aparelho	Recusado. Três tabelas, três controladores e três telas para três coisas que fazem o mesmo: relatar um evento que pode virar um caso. Triplica o custo de manutenção e triplica a chance de as regras divergirem.
Um gateway MQTT com fila de mensagens	Recusado **para esta fase**. É o desenho correto para operação real, com milhares de aparelhos, e acrescenta infraestrutura que não pode ser demonstrada num trabalho acadêmico sem virar encenação. Declarado como próximo passo na seção 6.3.
Um barramento único sobre HTTP, com discriminador de origem	**Adotado.** Descrito na seção 4.4.

A segunda linha merece uma palavra a mais, porque ela é a alternativa que um avaliador legitimamente cobraria. Em produção, dispositivos de campo não falam HTTP diretamente com a aplicação: falam com um intermediário que absorve instabilidade de rede, enfileira o que não pôde ser entregue e desacopla o ritmo do aparelho do ritmo do servidor. O desenho adotado aqui **não impede** essa evolução, porque o contrato de entrada permanece o mesmo: trocar o transporte significa trocar quem chama a rota, e não reescrever a regra.

4.2.4 Por que não Big Data

O enunciado desta fase cita **Big Data** entre os conhecimentos a aplicar, e esta seção registra por que ele não foi empregado. Não usar uma tecnologia é uma decisão tão sujeita a justificativa quanto usá-la, e a justificativa aqui é de escala.

A escala está medida. A base sobre a qual esta fase trabalha tem 190 denúncias e 106 eventos de dispositivos, e a janela analítica mais larga da Parte 4 recorta 160 registros. Nesses patamares, volume, variedade e velocidade são plenamente atendidos por um banco relacional com índices, e é o que a Parte 4 demonstra: as seis consultas respondem sobre a base inteira, e o agrupamento de focos custa O(k log k) sobre a janela. Introduzir armazenamento distribuído, processamento em lote ou um motor de fluxo contínuo neste ponto acrescentaria infraestrutura sem acrescentar resposta, e a seção 4.2.3 já registra o mesmo critério aplicado a outra escolha: adotar a tecnologia mais pesada antes de o problema pedir é gastar complexidade que alguém terá de manter.

Isso não quer dizer que a questão não exista, e sim que ela não é desta entrega. O barramento armazena **cada leitura recebida**, inclusive as que não violam o perímetro, e essa é exatamente a classe de dado que cresce com o tempo e não com a ocorrência: um parque de tornozeleiras em operação contínua produz linhas em ritmo constante, haja ou não incidente. Com os quatro totens e os aparelhos desta entrega isso é irrelevante. Com um parque real, é o primeiro lugar onde o modelo relacional pediria revisão, e aí a discussão sobre particionamento, política de retenção e armazenamento distribuído passa a ser pertinente. Registrar a fronteira é mais honesto do que atravessá-la para exibir a tecnologia.

4.3 A Inteligência Artificial que já roda no projeto

O enunciado pede explicitamente: *caso exista no projeto, já implementada e rodando, uma funcionalidade de IA ou IoT, explicar e justificar*. Esta seção cumpre essa exigência.

4.3.1 O que o VigIA faz

O VigIA é o componente de análise do Protege+, implementado em `service/VigiaService.java` e exposto em `controller/VigiaController.java`. Ele recebe o relato de uma denúncia e devolve três coisas: uma **urgência** em quatro níveis, um **resumo** para a triagem e a **origem da análise**.

4.3.2 O desenho de dois caminhos, e por que ele existe

O VigIA opera em dois modos:

- quando a variável `GEMINI_API_KEY` está configurada, ele consulta o modelo Gemini 1.5 Flash e declara `origem = GEMINI`;
- quando não está, ele aplica um conjunto de regras explícitas e declara `origem = REGRAS`.

Essa dualidade **não é um plano de contingência**, é uma decisão de projeto com três motivos.

O primeiro é de disponibilidade. Um canal de denúncias que para de classificar porque um serviço externo está fora do ar transfere ao cidadão o custo de uma indisponibilidade que não é dele.

O segundo é de verificabilidade acadêmica. O trabalho precisa ser executável por quem corrige, sem chave de API e sem custo. O modo por regras garante que a demonstração funcione ao descompactar o arquivo.

O terceiro, e mais importante, é de **prestação de contas**. O campo `origem_analise` é gravado em cada denúncia. Isso significa que, olhando um caso, é possível saber se a urgência dele foi atribuída por um modelo ou por uma regra legível. Um sistema que classifica sem dizer como classificou é uma caixa-preta com autoridade sobre atendimento, e a orientação corrente de governança de IA converge para a exigência de que decisões automatizadas sejam explicáveis a quem elas afetam.

4.3.3 As regras, escritas por extenso

O modo por regras atribui gravidade por tipo e eleva para o nível crítico quando o texto contém marcadores de imediatismo ou de arma:

```
GRAVIDADE = { violencia: 5, abuso: 5, assedio: 3, discriminacao: 3, outros: 2 }

PALAVRAS = arma, faca, agora, socorro, ameaca, ameaça,
           sangue, espancad, refem, refém

CRITICA  quando gravidade >= 5  E  o relato contém alguma PALAVRA
ALTA     quando gravidade >= 5
MEDIA    quando gravidade >= 3  OU  o relato contém alguma PALAVRA
BAIXA    nos demais casos
```

O terceiro degrau merece atenção porque é o menos óbvio: um relato de tipo *outros*, que sozinho valeria gravidade 2, sobe para MÉDIA se o texto contiver um marcador de imediatismo. A regra reconhece que **o tipo declarado é a informação menos confiável do formulário**, e que o texto pode contradizê-lo.

Essas regras são simples ao ponto de parecerem ingênuas, e essa é a intenção. Elas podem ser lidas por um servidor público sem formação técnica, contestadas caso a caso e corrigidas sem retreinar nada.

4.3.4 Como a IA existente se conecta ao IoT desta fase

Este é o ponto de junção entre a funcionalidade herdada e a nova, e ele é uma decisão deliberada: **os dispositivos não classificam.**

Um aparelho relata um evento. O servidor monta um texto descrevendo o que aconteceu e entrega esse texto ao VigIA, que o classifica com o mesmo critério aplicado a qualquer relato digitado por uma pessoa.

A alternativa, deixar o aparelho declarar a própria urgência, foi recusada. Se um dispositivo pudesse dizer *isto é crítico*, a ordem da fila de atendimento passaria a ser definida por **quem tem o aparelho**, e não pela gravidade do caso. Bastaria um equipamento mal configurado, ou deliberadamente adulterado, para que um evento irrelevante ocupasse o topo da fila de uma central.

A seção 5.2.5 apresenta a consulta SQL que **verifica essa afirmação no banco**: se a origem determinasse a urgência, cada origem teria um valor único, e os dados mostram que o totem produz as quatro urgências.

4.4 Arquitetura: um barramento, três origens

4.4.1 A decisão central

As três origens escrevem na mesma tabela, `EVENTO_IOT`, distinguidas pela coluna `origem`, que assume os valores `SOS`, `GEOFENCE` e `TOTEM`.

A justificativa é que **as três fazem a mesma coisa**: relatam um evento físico que o servidor avalia e que pode ou não virar um caso. O que varia entre elas é a regra de avaliação, não a natureza do dado.

O ganho é concreto e mensurável. Existe **um** controlador em vez de três, **uma** tela de acompanhamento em vez de três, **uma** trilha de auditoria e **uma** rota de consulta. Acrescentar uma quarta origem, uma câmera com botão de emergência em ponto de ônibus, por exemplo, custa uma constante e um método, e não um subsistema.

O custo, que precisa ser dito, é que a tabela tem colunas que não se aplicam a todas as origens: um totem não reporta bateria, e um botão de pânico não tem processo judicial vinculado. Essa é a forma de herança conhecida como *tabela única*, e a crítica clássica a ela é justamente a presença de colunas nulas (FOWLER, 2002). A decisão foi consciente: com três origens e sete colunas comuns contra duas específicas, o custo da nulidade é menor do que o custo da fragmentação.

4.4.2 O que o discriminador carrega

```
EVENTO_IOT
  id                identidade
  origem            SOS | GEOFENCE | TOTEM
  dispositivo       identificador OPACO do aparelho
  latitude          coordenada, quando aplicável
  longitude         coordenada, quando aplicável
  bateria           percentual, quando o aparelho reporta
  recebido_em       instante de recebimento no servidor
  situacao          ACEITO | DUPLICADO | DESCARTADO
  protocolo_gerado  protocolo do caso aberto, quando houver
  detalhe           o que o servidor decidiu, em texto curto
```

Duas colunas merecem comentário.

A coluna `dispositivo` guarda um identificador **opaco**. Ele não é o número de série do fabricante, não é documento de pessoa e não permite chegar a ninguém sem consultar um cadastro que vive fora desta tabela. Isso é minimização de dados no sentido do artigo 6°, inciso III, da LGPD (BRASIL, 2018a): o barramento guarda o necessário para operar, e não o máximo que poderia guardar.

A coluna `situacao` guarda a **decisão do servidor**, e não o estado do aparelho. É ela que permite à tela responder a pergunta que interessa a quem opera: não *o que chegou*, mas *o que foi feito com o que chegou*.

4.4.3 Por que o protocolo é texto, e não chave estrangeira

Esta é a decisão mais discutível do capítulo, e por isso ela é declarada em vez de escondida.

A coluna `protocolo_gerado` guarda o protocolo da denúncia como texto. Não há chave estrangeira de `EVENTO_IOT` para `DENUNCIA`, embora o modelo físico declare dez chaves estrangeiras em outras relações e o projeto trate integridade referencial como valor.

A razão é de **ciclo de vida**. O evento de dispositivo é um registro de telemetria: ele descreve algo que um aparelho enviou num instante, e essa afirmação permanece verdadeira para sempre. A denúncia, por sua vez, tem ciclo de vida próprio e pode ser **logicamente excluída**, com motivo registrado, conforme a US35 e o direito de eliminação previsto no artigo 18 da LGPD.

Com chave estrangeira rígida, haveria duas saídas ruins: impedir a exclusão da denúncia enquanto existisse evento apontando para ela, o que subordinaria um direito do titular a um registro de telemetria; ou propagar a exclusão para o evento, apagando o registro de que um aparelho enviou algo, o que corromperia a auditoria do parque de equipamentos.

A escolha foi manter os dois lados independentes. O custo é que o protocolo pode apontar para uma denúncia excluída, e a tela precisa tratar esse caso. O benefício é que **nem o direito do titular nem a integridade da telemetria ficam reféns um do outro.**

Registra-se a alternativa não adotada: uma tabela de ligação com exclusão em cascata apenas do vínculo, preservando os dois registros. Ela é mais correta formalmente e foi recusada por acrescentar uma tabela e uma junção a um caminho que é percorrido a cada leitura da tela, sem alterar o comportamento observável.

4.5 Origem A: botão de pânico

4.5.1 O que o aparelho envia

```
POST /api/iot/acionamento
X-Device-Key: <chave do parque de dispositivos>

{ "dispositivo": "SOS-8812", "lat": -23.5108, "lng": -46.8760, "bateria": 37 }
```

O exemplo carrega quatro campos. O contrato completo, em `dto/AcionamentoSosRequest.java`, aceita dois opcionais a mais, `cidade` e `estado`, usados quando o parque de dispositivos opera fora do município padrão; omitidos, o servidor assume Barueri e São Paulo.

O que **não** está no contrato é tão relevante quanto o que está: não há texto, não há tipo de denúncia e não há urgência. O aparelho informa quem é, onde está e quanta bateria tem.

4.5.2 O que o servidor faz

O método `registrarAcionamento` de `EventoIotService` executa, nesta ordem: verifica a chave do dispositivo; grava o evento; consulta o último acionamento aceito daquele aparelho; se estiver dentro da janela de repique, marca como duplicata e encerra; caso contrário, monta o relato, chama `DenunciaService` para abrir o caso e registra na trilha de auditoria.

O relato montado pelo servidor é este:

> Pedido de socorro enviado agora por botão de pânico físico. Dispositivo SOS-8812. Coordenada informada pelo aparelho: -23.510800, -46.876000. Bateria do aparelho em 37%. Acionamento sem texto: a pessoa não teve condição de escrever.

Uma ressalva de exatidão sobre a citação acima: o texto é gravado **sem acentuação**, por convenção herdada das fases anteriores, e aparece aqui acentuado apenas por legibilidade. Essa convenção alcança quase todo o código de produção, e é dívida declarada na seção 6.2.

As palavras *socorro* e *agora* estão nesse texto e disparam a regra de urgência crítica do VigIA. Isso precisa ser dito com clareza para não parecer manipulação do classificador: **elas descrevem o que um botão de pânico é**. Um acionamento de botão de emergência é, por definição, um pedido de socorro imediato, e o classificador está lendo o texto, não a origem. A última frase do relato é igualmente deliberada: ela informa a quem vai atender que a ausência de descrição é uma característica do canal, e não desinteresse de quem acionou.

4.5.3 A regra de repique

Um novo acionamento do mesmo aparelho dentro de **120 segundos** é gravado como `DUPLICADO` e vinculado ao protocolo anterior.

A justificativa é comportamental: **um botão apertado três vezes em dez segundos é uma pessoa com medo, não três ocorrências.** Sem a regra, uma central receberia três casos críticos idênticos, e o efeito prático de três alertas para o mesmo fato é a central aprender a desconfiar do alerta.

O evento duplicado não é descartado. Ele é gravado, vinculado ao caso original, e aparece na tela. A distinção importa: descartar apagaria a informação de que a pessoa acionou de novo, que é justamente um sinal de que a situação piorou.

A seção 5.2.3 apresenta a consulta que **audita esse número com dado**, em vez de defendê-lo com argumento.

4.5.4 Uma armadilha de localidade, e por que ela é grave

A coordenada é formatada por um método próprio:

```
public static String coordenada(Double valor) {
    return valor == null ? "" : String.format(Locale.ROOT, "%.6f", valor);
}
```

O `Locale.ROOT` não é zelo excessivo. `String.format` usa a localidade padrão da máquina virtual; numa máquina configurada em português, o resultado seria `-23,510800`, com vírgula. Uma coordenada com vírgula quebra qualquer consumidor que espere o formato WGS 84, e o texto do relato deixa de ser legível por sistemas parceiros.

O que torna esse defeito perigoso é o ambiente em que ele se manifesta: ele **não aparece** em máquina configurada em inglês, que é a configuração usual de servidores e de ambientes de integração contínua. Ele apareceria em produção, no Brasil. O defeito foi encontrado por um teste que falhou nesta fase, e a

correção veio acompanhada de um teste de regressão que força a localidade brasileira e a restaura ao final. O caso está no Anexo D, item 1.

4.6 Origem B: geofencing de medida protetiva

4.6.1 O que o aparelho envia, e o que ele recebe de volta

```
POST /api/iot/telemetria
X-Device-Key: <chave>

{ "tornozeleira": "TZ-4471", "lat": -23.5041, "lng": -46.8760, "bateria": 64 }

resposta 202:
{ "situacao": "ACEITA", "alertaAberto": true, "protocoloGerado": "#2026-90032" }
```

4.6.2 O perímetro pertence ao servidor

Esta é a decisão de segurança mais importante da fase.

A tornozeleira informa **apenas onde ela está**. Quem sabe onde mora a pessoa protegida, qual é o raio determinado e se a medida continua válida é o banco de dados, na tabela `ZONA_EXCLUSAO`.

A alternativa seria enviar o perímetro ao aparelho e deixar que ele mesmo avaliasse a violação. Isso pouparia tráfego e processamento, e é como muitos sistemas de cerca virtual funcionam. Seria também **entregar a coordenada da vítima a um equipamento que está, por definição, na posse da pessoa contra quem a medida foi expedida.**

A mesma preocupação explica um detalhe do contrato de resposta: `TelemetriaResponse` devolve se um alerta foi aberto, e **não devolve a distância nem o raio**. Um aparelho que soubesse a distância poderia ser usado para localizar quem a medida existe para proteger, bastando andar e observar o número diminuir. Essa é uma aplicação direta de minimização: a resposta carrega o mínimo necessário para o aparelho saber que a leitura foi aceita.

4.6.3 Haversine, e por que não Pitágoras

A distância entre a posição informada e o ponto protegido é calculada pela fórmula de Haversine (SINNOTT, 1984), com raio da Terra de 6.371.000 metros.

A alternativa seria aplicar o teorema de Pitágoras diretamente sobre as diferenças de latitude e longitude. Isso trataria a superfície da Terra como um plano e produziria erro crescente com a distância e com o afastamento do equador. Nas centenas de metros de um perímetro de medida protetiva, a diferença é pequena; a fórmula errada continuaria sendo a escolha errada, e o custo de acertar é o mesmo.

Um erro correlato foi cometido e corrigido no simulador, e vale registro porque ele mostra o mesmo equívoco na entrada em vez de na saída: os botões de deslocamento usavam o mesmo divisor para latitude e longitude. Um grau de latitude vale cerca de 111.195 metros em qualquer lugar; um grau de

longitude encolhe conforme se afasta do equador e, na latitude de Barueri, vale cerca de 101.971 metros. O passo lateral era 9% maior do que o anunciado no botão.

4.6.4 A memória de estado, e o custo de não tê-la

A tabela `ZONA_EXCLUSAO` guarda duas colunas de estado: `dentro_do_perimetro` e `ultimo_alerta_em`.

Sem a primeira, cada leitura recebida enquanto o monitorado permanece dentro do perímetro abriria um caso novo. Com telemetria de trinta em trinta segundos, uma hora de violação viraria 120 denúncias do mesmo fato.

A segunda sustenta o intervalo mínimo de **600 segundos** entre dois alertas da mesma medida. O número é maior que o do botão de pânico por uma razão de natureza: o botão é apertado por uma pessoa, e a telemetria chega sozinha, no ritmo do equipamento.

O evento também distingue **entrada** de **permanência** no perímetro, e o texto entregue ao VigIA registra qual dos dois ocorreu. São situações operacionalmente diferentes: entrar pode ser trajeto; permanecer é intenção.

4.6.5 O que o relato entregue ao VigIA não contém

> Descumprimento de medida protetiva detectado agora por monitoração eletrônica. Entrada no perímetro de exclusão. Processo 0007441-88.2026, equipamento TZ-4471. Evento gerado pelo servidor a partir da telemetria, sem intervenção humana.

Não há nome, não há endereço e **não há a coordenada da pessoa protegida**. Há o número do processo, que é informação pública do ato judicial, e o identificador opaco do equipamento.

Existe um teste automatizado que verifica essa ausência, e ele é mais cuidadoso do que parece: além de varrer o texto procurando a assinatura de uma coordenada, ele **cola uma coordenada real no texto e confirma que a varredura reprova**. Sem essa segunda parte, um padrão de busca quebrado passaria calado para sempre, já que um teste que só sabe dizer *não encontrei* não distingue *está limpo* de *não sei procurar*.

4.7 Origem C: totem público

4.7.1 O totem não diz onde está

Esta é a assimetria mais interessante entre as três origens, e ela é deliberada.

O botão de pânico informa a própria coordenada, porque ele anda junto com a pessoa e é o único que sabe onde ela está. O totem, ao contrário, **não envia coordenada alguma**: o lugar do registro vem do cadastro da instalação, na tabela `PONTO_TOTEM`.

O motivo é que um aparelho parafusado numa parede, em local público, está exposto a acesso físico. Se ele pudesse declarar a própria posição, alguém com acesso ao equipamento faria denúncias aparecerem

em qualquer endereço do país. O mapa do painel, que é o que orienta o deslocamento das equipes, passaria a ser preenchido por quem tem uma chave de fenda.

A mesma tabela carrega a coluna `ativo`. Desligar um totem é ato administrativo, não exclusão: um equipamento retirado para manutenção precisa parar de aceitar registros sem que os casos já abertos por ele percam a referência do lugar onde foram feitos.

4.7.2 A janela de repique é seis vezes menor, e isso é uma premissa invertida

No botão de pânico, a janela é de 120 segundos. No totem, é de **20 segundos**.

A diferença não é calibragem fina, é inversão de premissa. O botão pertence a uma pessoa: o mesmo aparelho acionado duas vezes em dois minutos é a mesma pessoa. O totem é equipamento público com fila: **duas pessoas diferentes usando o mesmo aparelho em dois minutos é o comportamento esperado, não a anomalia.**

Herdar a janela longa faria a denúncia de quem chegou depois desaparecer dentro do caso de quem veio antes. Vinte segundos cobrem o toque duplo no botão de enviar e nada além disso. Há teste automatizado que compara as duas janelas sobre o mesmo intervalo de 60 segundos e verifica que uma classifica como repique e a outra não.

4.7.3 Registro sempre anônimo

O `RegistroTotemRequest` não tem campo de identificação, e essa ausência é requisito, não esquecimento.

Quem usa um totem está de pé num saguão, com a fila atrás. Pedir nome ali seria pedir que a pessoa se identifique em voz alta, ou por escrito numa tela visível, diante de quem estiver esperando a vez. O canal já oferece identificação voluntária no formulário web, onde a pessoa está sozinha; no equipamento público, a identificação é recusada por desenho.

4.7.4 Texto curto é recusado, texto longo é cortado

A assimetria é intencional.

Um relato com menos de quinze caracteres úteis não descreve fato algum e não dá o que apurar. Ele é recusado, e **o fragmento não é gravado**: guardar um pedaço de texto que não serve para nada seria guardar um dado sensível sem finalidade, o que contraria o princípio da necessidade previsto na LGPD.

Um relato acima do limite de gravação é **cortado, não recusado**. Recusar obrigaria a pessoa a reescrever de pé, na fila, e o início do relato é justamente onde ela conta o que houve.

O texto também passa por higienização antes da medição: um teclado de saguão produz teclas presas, quebras de linha de leitor de tela e caracteres de controle. A validação de tamanho conta o texto **já higienizado**, senão quinze espaços passariam pela porta.

4.7.5 O protocolo na tela, e um conflito sem solução boa

A resposta ao totem devolve o protocolo e, junto, o número de segundos que a tela pode exibi-lo.

O conflito é real e vale ser exposto: o protocolo é a chave de leitura da denúncia, e a pessoa precisa dele para acompanhar o caso. Num monitor de saguão, ele fica visível para a fila inteira.

Não há resposta boa para esse conflito, apenas a menos ruim: **mostrar, avisar que será apagado, e apagar**. O prazo adotado é de 45 segundos. A alternativa de não exibir o protocolo foi recusada porque deixaria a pessoa sem como acompanhar o que denunciou, o que é pior do que a exposição temporária de um código que, por si só, não identifica ninguém.

4.8 Os dados utilizados

O enunciado pede que os dados sejam explicados. Esta seção os declara por origem, com a classificação de sensibilidade e o motivo de cada um existir.

Dado	Origem	Natureza	Por que é coletado
Identificador do aparelho	As três	Pseudônimo, opaco	Distinguir equipamentos e aplicar a regra de repique
Latitude e longitude	SOS e GEOFENCE	Pessoal, sensível por contexto	Orientar o deslocamento da equipe e avaliar o perímetro
Bateria	SOS e GEOFENCE	Técnico	Detectar equipamento prestes a silenciar
Instante de recebimento	As três	Técnico	Aplicar as regras de janela e ordenar a operação
Texto do relato	TOTEM	Pessoal, sensível	É a denúncia em si
Tipo declarado	TOTEM	Categórico	Orientar a triagem, com normalização no servidor
Processo judicial	GEOFENCE	Público	Vincular o alerta à medida que o originou

Quatro decisões de coleta merecem justificativa expressa.

O totem não coleta coordenada. Justificado em 4.7.1.

O identificador do aparelho é opaco. Ele não permite chegar a uma pessoa sem consulta a um cadastro que vive fora deste barramento, o que aplica o princípio da minimização do artigo 6°, inciso III, da LGPD (BRASIL, 2018a).

A coordenada da pessoa protegida não é replicada no evento. Ela existe em `ZONA_EXCLUSAO` e não é copiada para `EVENTO_IOT` nem para o texto da denúncia. Um dado sensível replicado em três lugares tem três chances de vazar.

O tipo declarado pelo totem não é confiável, e o sistema não finge que é. O valor chega de um cliente que pode ser alterado. Em vez de recusar o registro por causa de um campo de classificação, o que perderia o relato de uma pessoa por um detalhe de metadado, um tipo desconhecido é normalizado para `outros` e a urgência sai do VigIA lendo o texto.

4.9 Integração com o sistema existente

4.9.1 O caminho de um evento até virar caso

```
aparelho
   |  POST /api/iot/{acionamento|telemetria|totem}  +  X-Device-Key
   v
IotController ......... verifica a chave, valida o corpo
   |
   v
EventoIotService ...... aplica a regra da origem (repique, perímetro, relato)
   |                    grava o evento em EVENTO_IOT
   |
   +--> DenunciaService ..... cria a denúncia, gera protocolo
   |          |
   |          +--> VigiaService ..... classifica urgência pelo TEXTO
   |
   +--> AuditoriaService .... registra na trilha encadeada
   |
   v
painel do órgão ....... Dispositivos de campo, Kanban, Mapa, Relatório
```

O ponto a observar é que **o evento não cria uma estrutura paralela**. Ele desemboca em `DenunciaService`, o mesmo serviço usado pelo formulário web. A partir dali, um caso aberto por botão de pânico é indistinguível, em tratamento, de um caso digitado por uma pessoa: aparece no Kanban, entra na fila de prioridade, conta na estatística e pode receber evidências.

Essa uniformidade é o principal benefício arquitetural da fase, e ela só foi possível porque a decisão de não deixar o dispositivo classificar manteve o caminho único.

4.9.2 A tela de acompanhamento

A tela **Dispositivos de campo**, no painel do órgão, é composta por `js/dispositivos.js` e `css/dispositivos.css` e consome duas rotas de leitura.

Ela responde a pergunta que interessa a quem opera: não *o que chegou*, mas *o que o servidor decidiu sobre o que chegou*. A última coluna da tabela mostra exatamente isso, e as três situações têm tratamento visual distinto.

A tela se repinta a cada oito segundos enquanto está aberta e **para o intervalo quando o usuário sai dela**, para o painel não consultar a API de uma tela que ninguém está olhando.

4.9.3 A ausência deliberada de modo offline

O restante do painel cai para o `localStorage` quando a API está fora, recurso entregue na US22 e ampliado no modo de demonstração offline do Enterprise Challenge. **As duas telas novas desta fase não têm esse plano B**, e a ausência é decisão, não omissão.

O plano B é legítimo para denúncias porque aquelas denúncias foram mesmo digitadas por alguém: exibi-las sem API é mostrar dado real de outra fonte. Um evento de dispositivo, não. Inventar um acionamento de botão de pânico seria **fabricar exatamente a evidência que a tela existe para exibir**, e uma central que aprende a ver alertas que não aconteceram deixa de acreditar nos que aconteceram.

O mesmo raciocínio vale para os focos de reincidência, com um agravante: um foco inventado é uma acusação inventada sobre um lugar real.

Sem API, as duas telas dizem que estão sem API.

4.10 Segurança e privacidade

4.10.1 Autenticação de dispositivo, e o que ela não é

As rotas de escrita exigem o cabeçalho `X-Device-Key`, comparado em tempo constante:

```
public static boolean chaveValida(String esperada, String recebida) {
    // comparação com acumulador XOR, sem retorno antecipado
}
```

Comparar com `equals` vazaria, pelo tempo de resposta, quantos caracteres iniciais um atacante acertou. O ataque é conhecido desde Kocher (1996) e o custo de evitá-lo é uma linha.

Uma ressalva de exatidão sobre o método: a comparação começa conferindo o tamanho das duas cadeias e retorna cedo quando eles diferem. Isso vaza o **comprimento** da chave esperada, e apenas ele. O vazamento é considerado aceitável porque o comprimento não estreita o espaço de busca de forma útil, e porque a alternativa, comparar até o fim de uma das cadeias, introduz o risco de leitura fora de faixa sem ganho prático.

A fragilidade descrita a seguir corresponde ao risco API2:2023, Broken Authentication, do catálogo de riscos de API da OWASP (OWASP, 2023).

Duas honestidades sobre esse mecanismo.

A primeira: **é uma proteção mais fraca do que a do restante da API**, que usa JWT com expiração. Uma chave compartilhada por um parque de dispositivos não identifica o aparelho individualmente e não expira sozinha. O desenho correto para produção é certificado por dispositivo, ou chave derivada por equipamento com rotação, e isso está declarado na seção 4.12.

A segunda, que é o que torna a fraqueza aceitável nesta fase: **as rotas de dispositivo só criam**. Não há rota que permita a um aparelho ler, alterar ou excluir. Quem obtiver a chave consegue poluir o

barramento, o que é ruim e detectável pela consulta da seção 5.2.2; não consegue extrair uma denúncia, e é essa a diferença que importa.

4.10.2 A superfície exposta pela política de CORS

A política de CORS declara origens explicitamente, em vez de usar curinga com credenciais. O caso do cabeçalho `X-Device-Key` está descrito em 2.6.2.

Uma consequência prática merece registro porque ela aparece na demonstração: a página do simulador **não funciona** se aberta por duplo clique, porque um arquivo lido do disco chega à API com origem `null`. Isso não é defeito a contornar. Liberar origem nula numa API que trabalha com credenciais abriria a porta para qualquer arquivo local disparar requisições autenticadas contra ela. A restrição foi mantida e a instrução de servir a página por HTTP está escrita na própria página.

4.10.3 O perímetro, o protocolo e a distância

As três decisões de privacidade específicas desta fase já foram justificadas em suas seções, e são reunidas aqui para conferência:

Decisão	Seção	Risco evitado
O perímetro fica no servidor	4.6.2	Entregar a coordenada da vítima ao monitorado
A resposta não devolve distância nem raio	4.6.2	Localização por aproximação sucessiva
O totem não informa a própria posição	4.7.1	Falsificação de endereço por acesso físico
O relato de violação não cita coordenada	4.6.5	Vazamento por texto entregue ao classificador
O totem não pede identificação	4.7.3	Exposição diante da fila
O protocolo tem prazo de tela	4.7.5	Leitura por terceiros no saguão

4.10.4 Perfis e auditoria

As duas telas novas seguem o controle de acesso da US24. Administrador, Gestor e Atendente acessam; **o perfil Auditor não**. A razão é que o Auditor fiscaliza o uso do sistema e não opera o atendimento, e o mesmo critério já separava dele o Kanban e a fila.

As rotas de leitura são anotadas com `@Auditavel(acao = CONSULTA_SENSIVEL)`, o que as coloca na trilha encadeada da US25. O critério de registrar acesso a dado sensível segue o controle 8.15 da ISO/IEC 27002:2022 (ISO/IEC, 2022) e a orientação da OWASP sobre quais eventos merecem registro (OWASP,

[s.d.]). A classificação desses eventos como tratamento de dado pessoal, com registro de quem acessou e quando, acompanha a orientação da ANPD para agentes de tratamento (ANPD, 2021).

Para a leitura de eventos, a justificativa é que a lista carrega coordenadas. Para os focos de reincidência, a justificativa é mais sutil e está na seção 5.10: **saber que vários relatos anônimos descrevem a mesma situação é uma informação sobre um lugar e sobre as pessoas que vivem nele**, e é informação que não existe em nenhuma denúncia isolada.

4.11 Benefícios da solução

Benefício	Como se manifesta	Onde verificar
Denúncia sem digitação	Um gesto físico abre um caso crítico	Seção 4.5
Detecção sem dependência da vítima	O servidor avalia o perímetro sozinho	Seção 4.6
Acesso sem aparelho próprio	Totem em equipamento público, anônimo	Seção 4.7
Redução de casos duplicados	Repique reconhecido em vez de multiplicado	Consulta 5.2.3
Classificação uniforme	O mesmo VigIA lê relato humano e relato de máquina	Consulta 5.2.5
Detecção de equipamento defeituoso	Concentração anômala de eventos num aparelho	Consulta 5.2.2
Custo marginal de nova origem	Uma constante e um método	Seção 4.4.1
Auditabilidade	Toda leitura sensível na trilha encadeada	Seção 4.10.4

Um benefício merece destaque por ser o menos evidente: **a decisão de não deixar o dispositivo classificar transformou uma afirmação de arquitetura em algo verificável no banco**. A consulta da seção 5.2.5 confirma que o totem produz as quatro urgências, o que só é possível se a classificação vier do texto. Uma afirmação de projeto que pode ser conferida com uma consulta vale mais do que a mesma afirmação repetida num documento.

4.12 Limitações declaradas

Esta seção existe porque o enunciado pede limitações, e porque um capítulo que só relata acertos descreve uma demonstração.

4.12.1 Limitações do que foi construído

A autenticação é por chave compartilhada. Justificada em 4.10.1 e mitigada pelo fato de as rotas de dispositivo só criarem. O correto é certificado por dispositivo com rotação, e não foi feito.

O transporte é HTTP síncrono. Não há fila nem reenvio. Um aparelho sem rede perde o evento. Em produção, isso exige um intermediário com buffer, discutido em 4.2.3 e declarado como próximo passo.

Não há verificação de integridade da mensagem. A chave autentica a origem, e nada assina o conteúdo. Um intermediário com acesso ao tráfego poderia alterar uma coordenada. Mitigado por HTTPS em produção, e não resolvido no nível da aplicação.

O parque de dispositivos não tem cadastro próprio. Só o totem tem tabela de instalação. Botões de pânico e tornozeleiras são reconhecidos pelo identificador que enviam, sem registro de a quem pertencem, quando foram entregues ou se ainda estão em uso.

Não há detecção de silêncio. O sistema reage ao que chega. Uma tornozeleira que para de transmitir, o que é exatamente o que aconteceria se fosse rompida, não gera alerta algum. Esta é a limitação mais séria da origem B, e a correção é conhecida: um processo que verifica periodicamente equipamentos sem telemetria recente.

A bateria é registrada e não é usada. O dado é coletado, aparece na consulta da seção 5.2.1 e não dispara nada. Um aparelho com 4% de bateria vai silenciar em breve, e o sistema não avisa ninguém.

4.12.2 Limitações do que não foi construído

Não há hardware. Os três aparelhos são simulados por uma página que fala o mesmo protocolo. A seção 4.13 discute o que essa substituição prova e o que ela não prova.

Não há integração com órgão de segurança pública. Um alerta de descumprimento de medida protetiva abre um caso no painel do Protege+ e não aciona ninguém fora dele. Em operação real, esse é o passo que falta para a funcionalidade ter efeito.

Não há validação jurídica do modelo de zona de exclusão. A tabela guarda um ponto e um raio. Medidas protetivas reais podem fixar perímetros sobre múltiplos endereços, com vigência e condições específicas, e o modelo atual não expressa isso.

4.13 O simulador de dispositivos

4.13.1 Por que ele existe

A Fase 6 pede dispositivos em campo, e não há tornozeleira nem totem para instalar num trabalho acadêmico. Diante disso, há duas saídas.

A primeira é encenar: gravar um vídeo apontando para um objeto e afirmar que ele é o dispositivo. A segunda é construir **o lado do servidor inteiro e de verdade** e substituir o aparelho por uma página que fala o mesmo protocolo: mesma rota, mesmo cabeçalho de autenticação, mesmo corpo JSON.

A segunda foi adotada. O que ela prova é preciso: **trocar a página por um aparelho real não muda uma linha do back-end.** O que ela não prova também precisa ser dito: nada sobre consumo de energia, alcance de rádio, resistência física do equipamento ou comportamento em rede instável.

4.13.2 Por que ele não é parte do produto

O simulador é instrumento de teste e está identificado como tal na própria página, com selo na barra superior e no rodapé.

O acesso a ele fica junto ao atalho da versão em React do formulário, no fim da página pública: os dois são atalhos de avaliação, e nenhum dos dois faz parte do fluxo de quem denuncia. O que a página nunca teve, e não deve ter, é um botão de *acionar pânico* solto dentro do canal real, que seria um alarme falso pendurado num sistema de emergência.

4.13.3 Uma decisão de projeto que o simulador obrigou a explicitar

O simulador tem botões que levam a tornozeleira a pontos marcados no mapa, entre eles um em frente ao endereço protegido. Isso parece contradizer a seção 4.6.2, que diz que o aparelho não pode conhecer o perímetro.

A contradição é aparente, e desfazê-la esclarece a regra. Quem opera o simulador é uma **pessoa**, não o aparelho. Quem carrega uma tornozeleira sabe perfeitamente onde mora a pessoa protegida: é exatamente por isso que a medida protetiva foi expedida. O que o **aparelho** não pode saber é o raio e a distância, e a resposta da API continua sem devolver nenhum dos dois.

A primeira versão do simulador errava nisso, e o erro é instrutivo: os botões diziam *aproximar* e *afastar*, o que exigiria que a página soubesse **para onde** aproximar, e portanto que ela calculasse a direção da zona. A correção não foi inverter o sinal, foi separar o que o aparelho sabe do que a pessoa sabe.

4.14 Síntese da Parte 3

Exigência do enunciado	Onde foi atendida
Identificar problema ou oportunidade	Seção 4.1, três problemas distintos

Exigência do enunciado	Onde foi atendida
Escolher a tecnologia mais adequada	Seção 4.2, IoT
Justificar a escolha	Seções 4.2.1 a 4.2.3, com três alternativas recusadas
Explicar os dados utilizados	Seção 4.8
Explicar a integração ao sistema	Seção 4.9
Apresentar benefícios	Seção 4.11
Apresentar limitações	Seção 4.12, nove limitações declaradas
Dar início à implementação	Seções 4.5 a 4.7, três origens funcionando
Explicar a IA já existente	Seção 4.3

PARTE 4: Análise e eficiência dos dados

> **O que o enunciado pede**
>
> *Aprimore a forma como o HackGov armazena, consulta, organiza ou processa informações. Análise com SQL: criar consultas com agrupamentos, totais ou métricas; utilizar ao menos uma subconsulta ou função avançada estudada; apresentar os resultados obtidos; explicar quais informações relevantes podem ser extraídas dessas consultas. Estruturas de Dados e Algoritmos: identifique uma situação do HackGov que envolva busca, organização, ordenação ou relacionamento de dados. Utilize conceitos como árvores, busca, ordenação, grafos e análise de complexidade com Notação Big O. Explique o problema, justifique a solução escolhida e demonstre sua aplicação no projeto.*

5.1 Método da análise com SQL

5.1.1 O critério de escolha das consultas

As seis consultas de `database/05_consultas_fase6.sql` não foram escolhidas para cobrir a lista de recursos de SQL pedida pelo enunciado. Elas foram escolhidas para **responder a seis perguntas que a operação faz e que ninguém consegue responder olhando a tela**. Os recursos exigidos aparecem porque a pergunta pediu, e não o contrário.

Essa ordem importa. Uma consulta escrita para exibir `ROLLUP` produz uma tabela correta que não informa nada; uma consulta escrita para responder *quanto cada origem produz e o que o servidor decidiu* precisa de subtotais, e aí o `ROLLUP` deixa de ser enfeite.

O quadro abaixo registra a pergunta de cada consulta e o recurso que ela exigiu.

#	Pergunta de operação	Recurso
1	Quanto cada origem produz, e o que foi decidido?	`GROUP BY ... ROLLUP` com `GROUPING`
2	O volume está distribuído ou preso em poucos aparelhos?	`RANK`, `SUM OVER`, CTE
3	A janela de repique de 120 s está bem calibrada?	`LAG`
4	Quais pontos de instalação estão acima da média?	Subconsulta no `HAVING`
5	O aparelho decide a urgência? O dado confirma?	Junção com agrupamento
6	De tudo que cada origem envia,	Métricas derivadas

#	Pergunta de operação	Recurso
	quanto vira caso?	

5.1.2 Portabilidade

O script foi escrito para rodar no Oracle do modelo físico e no H2 embutido usado na demonstração. Onde os dois divergem, a diferença está anotada no comentário da própria consulta. Duas divergências existem: a subtração de instantes, que no Oracle produz dias e no H2 exige `DATEDIFF`; e o tipo booleano, que no Oracle é `NUMBER(1)`. A sintaxe de `ROLLUP`, `GROUPING` e das funções analíticas usadas aqui é a da referência de linguagem do Oracle (ORACLE, [s.d.]), e é aceita pelo H2 sem alteração.

5.1.3 A base sobre a qual as consultas rodam

Os resultados apresentados vêm de **106 eventos de dispositivo e 65 denúncias abertas por eles**, entre 3 de abril e 14 de maio de 2026, produzidos pelo programa `database/gerar_carga_fase6.py` com semente 561601. Somadas às 125 denúncias herdadas das fases anteriores, a base tem **190 denúncias**.

A carga é de demonstração, declarada como tal, no mesmo regime do restante do `data.sql`. O que a torna utilizável em documento acadêmico é a **reprodutibilidade**: reexecutar o gerador produz exatamente os mesmos registros em qualquer máquina, de modo que qualquer número citado aqui pode ser conferido na origem por quem corrige (PENG, 2011).

5.2 As seis consultas, seus resultados e o que eles informam

5.2.1 Consulta 1: o barramento em uma tabela só

```
SELECT
    CASE WHEN GROUPING(origem)   = 1 THEN '>> TOTAL GERAL' ELSE origem   END AS
origem,
    CASE WHEN GROUPING(situacao) = 1 THEN '.. subtotal'    ELSE situacao END AS
situacao,
    COUNT(*)                AS eventos,
    COUNT(protocolo_gerado) AS casos_abertos,
    ROUND(AVG(bateria), 1)  AS bateria_media
FROM evento_iot
GROUP BY ROLLUP (origem, situacao)
ORDER BY GROUPING(origem), origem, GROUPING(situacao), situacao;
```

origem	situação	eventos	casos abertos	bateria média
GEOFENCE	ACEITO	36	11	49,3
GEOFENCE	DUPLICADO	2	0	60,5
GEOFENCE	*subtotal*	38	11	49,9
SOS	ACEITO	32	32	54,6

origem	situação	eventos	casos abertos	bateria média
SOS	DUPLICADO	8	8	49,8
SOS	*subtotal*	40	40	53,6
TOTEM	ACEITO	22	22	(sem)
TOTEM	DESCARTADO	6	0	(sem)
TOTEM	*subtotal*	28	22	(sem)
TOTAL GERAL		**106**	**73**	**51,8**

O que isto informa. Cento e seis eventos produziram 73 casos. A tornozeleira é a origem mais falante e a que menos abre caso, e esse é o comportamento correto: ela envia posição continuamente e o normal é estar fora do perímetro.

A bateria vazia no totem **não é dado faltando**: totem fica na tomada e não reporta bateria. Uma coluna vazia que significa *não se aplica* é diferente de uma que significa *não chegou*, e o modelo preserva essa diferença ao permitir nulo em vez de gravar zero. Gravar zero produziria uma média de bateria falsamente baixa e um alerta de equipamento prestes a silenciar que nunca silenciaria.

Por que o `ROLLUP`. Ele entrega, numa consulta só, as três granularidades que a gestão pede em momentos diferentes: detalhe por situação, subtotal por origem e total do parque. Sem ele seriam três consultas, e três consultas separadas correm o risco clássico de não fecharem entre si quando executadas em instantes diferentes.

Por que o `GROUPING`. As linhas de subtotal têm a coluna de situação nula. Sem `GROUPING`, seria impossível distinguir esse nulo do nulo que porventura existisse no dado, que é uma armadilha conhecida do `ROLLUP`.

5.2.2 Consulta 2: quais aparelhos concentram o volume

```
WITH por_aparelho AS (
    SELECT origem, dispositivo, COUNT(*) AS eventos
    FROM evento_iot WHERE situacao = 'ACEITO'
    GROUP BY origem, dispositivo
)
SELECT origem, dispositivo, eventos,
    RANK() OVER (PARTITION BY origem ORDER BY eventos DESC) AS posicao,
    ROUND(100.0 * eventos / SUM(eventos) OVER (PARTITION BY origem), 1) AS
pct_da_origem,
    ROUND(100.0 * SUM(eventos) OVER (PARTITION BY origem ORDER BY eventos DESC
          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
          / SUM(eventos) OVER (PARTITION BY origem), 1) AS pct_acumulado
FROM por_aparelho
ORDER BY origem, eventos DESC;
```

origem	dispositivo	eventos	posição	% da origem	% acumulado
GEOFENCE	TZ-4488	16	1	44,4	44,4
GEOFENCE	TZ-4471	10	2	27,8	72,2
GEOFENCE	TZ-4502	10	2	27,8	100,0
SOS	SOS-8840	10	1	31,2	31,2
SOS	SOS-8817	7	2	21,9	53,1
SOS	SOS-8823	6	3	18,8	71,9
SOS	SOS-8812	5	4	15,6	87,5
SOS	SOS-8831	4	5	12,5	100,0
TOTEM	TT-002	10	1	45,5	45,5
TOTEM	TT-001	6	2	27,3	72,7
TOTEM	TT-004	6	2	27,3	100,0

Por que a pergunta importa. A resposta muda a ação. Volume espalhado é demanda real e pede equipe. Volume concentrado em um ou dois aparelhos costuma ser outra coisa: defeito de hardware, botão sendo pressionado dentro do bolso, ou uma pessoa em situação muito pior que a média. As três leituras exigem operações diferentes, e por isso a pergunta precisa ser respondida antes de agir.

O que isto informa. Não há concentração patológica em nenhuma origem. No botão de pânico, dois dos cinco aparelhos somam 53,1%, o que é distribuição normal para cinco unidades. A leitura útil é a que **não** apareceu: nenhum aparelho sozinho acima de 50% da própria origem.

Uma observação sobre o `RANK`. TZ-4471 e TZ-4502 aparecem ambos na posição 2, e a posição 3 não existe naquela origem. Isso é o comportamento correto de `RANK`, que empata e pula. Usar `DENSE_RANK` produziria 1, 2, 2 e depois 3, o que seria adequado se a pergunta fosse *quantos níveis existem*; a pergunta aqui é *quantos aparelhos estão à frente*, e para ela o salto é a resposta certa.

Por que o percentual acumulado. Ele é o que responde de fato à pergunta da concentração. O percentual isolado diz quanto cada aparelho produz; o acumulado diz quantos aparelhos são necessários para explicar metade do volume, que é a forma operacional da pergunta.

5.2.3 Consulta 3: a janela de repique está bem calibrada?

Esta é a consulta mais importante do conjunto, porque ela **audita uma decisão de projeto com dado**, em vez de defendê-la com argumento.

```
SELECT faixa, COUNT(*) AS ocorrencias
FROM (
  SELECT CASE
```

```
           WHEN seg <  20 THEN 'a) ate 20 s . toque duplo'
           WHEN seg < 120 THEN 'b) 20 a 120 s . dentro da janela'
           WHEN seg < 300 THEN 'c) 120 a 300 s . logo acima da janela'
           ELSE                'd) acima de 300 s . evento independente'
         END AS faixa
  FROM (
    SELECT (CAST(recebido_em AS DATE) - CAST(anterior AS DATE)) * 86400 AS seg
    FROM (
      SELECT dispositivo, recebido_em,
             LAG(recebido_em) OVER (PARTITION BY dispositivo
                                    ORDER BY recebido_em) AS anterior
      FROM evento_iot WHERE origem = 'SOS'
    ) x WHERE anterior IS NOT NULL
  ) y
) z
GROUP BY faixa ORDER BY faixa;
```

faixa	ocorrências
a) até 20 s, toque duplo	4
b) 20 a 120 s, dentro da janela	4
c) 120 a 300 s, logo acima da janela	**6**
d) acima de 300 s, evento independente	21

Por que `LAG` e não uma autojunção. `LAG` traz o instante do evento anterior **do mesmo aparelho**, que é exatamente a informação que a regra usa em tempo de execução. A consulta é, portanto, a mesma pergunta do código feita ao banco. A alternativa com autojunção produziria o mesmo resultado com custo quadrático e com uma cláusula de correlação que esconderia a intenção.

O que isto informa. Dos 35 intervalos observados entre acionamentos do mesmo aparelho, 8 caem dentro da janela de 120 segundos e são tratados como repique.

Os **6 da faixa (c)** são o achado. Eles estão logo acima do corte e viraram caso novo. Seis ocorrências imediatamente acima do limite, contra oito dentro dele, é proporção alta o bastante para merecer investigação em operação real: ou são duas ocorrências distintas de fato, ou a janela está curta e a mesma pessoa está abrindo dois casos. **Com este volume não é possível decidir**, e a resposta honesta é essa.

O que a consulta entrega não é a resposta pronta: é o **instrumento** que passa a medir a janela continuamente, e o critério para revisá-la. Se, em operação real, a faixa (c) continuar nessa proporção, a janela está curta e casos duplicados estão passando; se nenhuma ocorrência chegar perto de 120 segundos, ela está larga e poderia engolir um acionamento legítimo.

5.2.4 Consulta 4: totens acima da média

```
SELECT p.dispositivo, p.local_instalacao, p.cidade,
       COUNT(*) AS registros,
       SUM(CASE WHEN e.situacao = 'ACEITO' THEN 1 ELSE 0 END) AS aceitos
FROM ponto_totem p
JOIN evento_iot  e ON e.dispositivo = p.dispositivo
```

```sql
WHERE p.ativo = TRUE
GROUP BY p.dispositivo, p.local_instalacao, p.cidade
HAVING COUNT(*) > (
    SELECT AVG(qtd) FROM (
        SELECT COUNT(*) AS qtd
        FROM evento_iot e2 JOIN ponto_totem p2 ON p2.dispositivo = e2.dispositivo
        WHERE p2.ativo = TRUE GROUP BY e2.dispositivo
    ) media_por_totem
)
ORDER BY registros DESC;
```

dispositivo	local de instalação	cidade	registros	aceitos
TT-002	Terminal Rodoviário de Barueri	Barueri	12	10

Por que a subconsulta, e não um número fixo. A média é recalculada a cada execução. Um limite fixo, do tipo *mais de 8 registros*, envelheceria em silêncio conforme o parque crescesse, e continuaria devolvendo uma resposta com cara de certa. Esta é a razão de a subconsulta estar aqui, e não o cumprimento de um requisito.

O que isto informa. A média por totem ativo é de 8,0 registros, e um único ponto está acima dela: o Terminal Rodoviário, com 12. TT-001 e TT-004 ficaram ambos em 6.

O par 12 registros contra 10 aceitos é o dado que orienta ação: dois registros foram recusados ali, e recusa em totem tem causa conhecida e tratável, relato curto demais, que é problema de instrução na tela, ou equipamento fora de serviço, que é problema de manutenção.

Vale registrar o que este resultado **não** autoriza a concluir. Um ponto acima da média entre três não estabelece padrão: com três observações, a média é frágil e um único ponto a desloca. A consulta é o instrumento correto e a base é pequena demais para a conclusão, e essa distinção precisa estar dita.

Uma correção registrada. A primeira versão desta consulta usava `COUNT(e.id)` em vez de `COUNT(*)`. As duas formas são equivalentes quando a coluna nunca é nula, e divergem quando ela pode ser. Numa verificação em banco carregado por script, sem a coluna de identidade preenchida, `COUNT(e.id)` devolvia zero e a consulta retornava vazia. O defeito está no Anexo D, item 8, e ele é um bom exemplo de consulta sintaticamente correta que responde à pergunta errada.

5.2.5 Consulta 5: o aparelho decide a urgência? O dado confirma?

```sql
SELECT e.origem, d.urgencia_ia, COUNT(*) AS casos,
       ROUND(AVG(d.score), 1) AS score_medio, MIN(d.protocolo) AS exemplo
FROM evento_iot e
JOIN denuncia   d ON d.protocolo = e.protocolo_gerado
WHERE e.situacao = 'ACEITO' AND e.protocolo_gerado IS NOT NULL
GROUP BY e.origem, d.urgencia_ia
ORDER BY e.origem, casos DESC;
```

origem	urgência	casos	score médio	exemplo
GEOFENCE	CRÍTICA	11	70,0	#2026-90033

origem	urgência	casos	score médio	exemplo
SOS	CRÍTICA	32	80,0	#2026-90001
TOTEM	MÉDIA	11	80,0	#2026-90044
TOTEM	BAIXA	5	80,0	#2026-90051
TOTEM	CRÍTICA	4	80,0	#2026-90045
TOTEM	ALTA	2	80,0	#2026-90054

O que esta consulta prova. A seção 4.3.4 afirma que o dispositivo não decide a urgência, e que quem classifica é o VigIA lendo o texto. Se a afirmação fosse falsa, cada origem teria um valor único, porque a origem é que estaria decidindo.

O totem produz **as quatro urgências**. O mesmo equipamento gera de BAIXA a CRÍTICA conforme o que a pessoa escreveu. É a prova de que a classificação vem do texto.

A leitura cuidadosa que a tabela exige. SOS e GEOFENCE saem sempre CRÍTICA, e isso pode ser confundido com o contrário do que a consulta mostra. Não é a origem que fixa a urgência: é que o texto que essas duas origens produzem **sempre** descreve um fato imediato. Um botão de pânico é, por definição, um pedido de socorro agora, e o classificador lê *agora* no relato como leria em qualquer texto digitado. A regra é a mesma; o que muda é a entrada. A prova disso está na própria tabela: se fosse a origem, o totem também teria valor único.

A coluna de score prova outra coisa, e é a informação mais interessante da tabela. As quatro linhas de TOTEM têm urgências diferentes e o **mesmo score**, 80. Isso não é defeito: **score e urgência medem coisas distintas**, e a tabela é a demonstração empírica disso.

O score vem do `ScoreService` e mede **quanto o formulário foi preenchido**; a urgência vem do VigIA e mede **gravidade**. Todos os registros de totem preenchem exatamente os mesmos campos, então pontuam igual, e ainda assim se separam em quatro urgências. As duas medidas são independentes por construção, e uma tabela em que uma varia e a outra não é a prova de que não há contaminação entre elas.

A diferença de 80 para 70 entre SOS e GEOFENCE tem a mesma natureza e também é uma decisão aparecendo no dado: o acionamento de pânico grava endereço aproximado, que vale 10 pontos de preenchimento, e o alerta de perímetro **não grava endereço algum**, por decisão de privacidade tomada na seção 4.6.2. Os 10 pontos de diferença são exatamente essa decisão, medida.

Uma afirmação de arquitetura que pode ser conferida no banco vale mais do que a mesma afirmação repetida num documento.

5.2.6 Consulta 6: aproveitamento por origem

```
SELECT origem, COUNT(*) AS total,
  SUM(CASE WHEN protocolo_gerado IS NOT NULL THEN 1 ELSE 0 END) AS geraram_caso,
```

```
  SUM(CASE WHEN situacao = 'DUPLICADO'  THEN 1 ELSE 0 END)      AS duplicados,
  SUM(CASE WHEN situacao = 'DESCARTADO' THEN 1 ELSE 0 END)      AS descartados,
  ROUND(100.0 * SUM(CASE WHEN protocolo_gerado IS NOT NULL THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 1) AS aproveitamento_pct
FROM evento_iot GROUP BY origem ORDER BY total DESC;
```

origem	total	geraram caso	duplicados	descartados	aproveitamento
SOS	40	40	8	0	100,0%
GEOFENCE	38	11	2	0	28,9%
TOTEM	28	22	0	6	78,6%

A leitura útil não é *quanto maior, melhor*. Cada origem tem um aproveitamento esperado diferente, e o que informa é o desvio dele.

SOS em 100% está no esperado. Ninguém aperta um botão de pânico por engano com frequência. Os 8 duplicados também contam como *geraram caso*, porque um repique fica vinculado ao caso anterior: ele não abre caso novo, e também não se perde.

GEOFENCE em 28,9% é a boa notícia da tabela. A tornozeleira reporta posição continuamente, e o normal é estar fora da área proibida. Um aproveitamento **alto** aqui seria o alarme, porque significaria perímetro violado na maior parte das leituras.

TOTEM em 78,6% com 6 descartes é o único número que pede ação. As duas causas de descarte em totem são tratáveis, e estão nomeadas na seção 5.2.4.

Por que `NULLIF`. Ele evita divisão por zero numa origem ainda sem eventos, o que aconteceria no primeiro dia de operação de um equipamento novo. É uma proteção para um caso que a base de demonstração não contém e a operação real conterá.

5.3 A carga determinística, e um erro de método que quase entrou

Esta seção registra um erro que foi cometido durante esta fase, percebido antes da entrega e corrigido. Ele está aqui porque o erro é instrutivo e porque omiti-lo tornaria os resultados da seção 5.2.3 menos confiáveis do que eles são.

O erro. A primeira versão do gerador de carga sorteava o intervalo entre acionamentos repetidos **dentro** da janela de 120 segundos, usando um sorteio uniforme entre 4 e 95 segundos. Parecia razoável: o objetivo era produzir repiques, e repique é o que acontece dentro da janela.

Por que isso invalidaria a consulta 3. Aquela consulta existe para auditar se a janela de 120 segundos está bem calibrada. Se os intervalos forem sorteados dentro da janela, a consulta conclui que a janela cobre tudo **por construção**. A faixa (c) viria vazia, e a conclusão *a janela está bem calibrada* seria uma consequência aritmética da forma como o dado foi gerado, e não um achado. Isso é raciocínio circular, e teria passado despercebido por quem lesse apenas o resultado.

A correção. O gerador passou a sortear o intervalo de uma distribuição exponencial de média 55 segundos, que é a forma usual de modelar tempo entre eventos de um mesmo processo, e a **aplicar depois** a regra dos 120 segundos, como o servidor faria. Quantos intervalos caem acima do corte passou a ser consequência da distribuição, e não escolha de quem escreveu o gerador. A carga também passou a refletir a consequência: um intervalo acima da janela gera **caso novo** no dado, e não um duplicado, porque é isso que o servidor faria.

O resultado dessa correção é a faixa (c) com 6 ocorrências, discutida em 5.2.3.

A lição, registrada porque vale além deste caso. Quando se gera o dado sobre o qual se vai medir, é preciso separar a **geração** da **regra**. Se a regra participa da geração, a medição verifica a própria geração, e não a regra.

5.4 O problema das estruturas de dados: focos de reincidência

5.4.1 A situação concreta

A central recebe denúncias majoritariamente anônimas. **Anônimo significa que não existe nome para agrupar.** Sem nome, o gestor vê uma lista plana onde seis relatos podem ser seis ocorrências distintas ou seis relatos da mesma situação em curso.

A diferença muda a resposta inteira. Seis ocorrências distintas são seis visitas independentes. Um foco único com seis relatos em três semanas é uma operação coordenada, e provavelmente urgente, porque a repetição em curto intervalo é ela própria um indicador de gravidade.

Até esta fase, o painel sabia **contar** denúncias e não sabia **relacioná-las**. A limitação estava declarada na seção 6.3 do documento da Fase 5.

5.4.2 Por que isso é um problema de estrutura de dados

A pergunta *quais denúncias descrevem a mesma situação* tem três características que a colocam fora do alcance de um filtro.

A relação é indireta. Duas denúncias podem pertencer ao mesmo fenômeno sem se parecerem diretamente, ligadas por uma terceira que está entre as duas no tempo.

A relação é temporal e tem fim. Um agrupamento não é permanente: ele começa, dura e termina. Um filtro não expressa duração.

O volume cresce. A base já tem 190 denúncias e uma operação real teria ordens de grandeza mais. Uma solução que compare todos os pares contra todos não é viável, e a seção 5.7 mostra que a solução ingênua é exatamente essa.

5.5 Por que não um GROUP BY

Esta é a pergunta que a solução precisa responder, porque a alternativa óbvia é agrupar por município e tipo no próprio SQL, o que custaria uma linha.

Um `GROUP BY` junta **tudo** o que compartilha a chave, do primeiro registro até o último. Duas denúncias do mesmo bairro separadas por oito meses cairiam no mesmo balde, e o resultado seria um número que só cresce e nunca informa nada. Pior: ele **pareceria** informar, porque *Barueri*, *violência*, *44 casos* é uma frase que soa como um achado.

Um foco não é um balde: é uma **cadeia no tempo**.

> A pertence ao mesmo foco que B se os dois estão próximos. B pertence ao mesmo de C pela mesma razão. Então A e C são o mesmo foco **ainda que estejam distantes entre si**, porque há continuidade entre eles. Quando a cadeia se interrompe, o foco terminou, e um relato posterior começa outro.

Essa relação, *ligado diretamente ou por intermediários*, é exatamente a definição de **componente conexo** de um grafo (CORMEN et al., 2022; HOPCROFT; TARJAN, 1973). Não existe cláusula de SQL padrão que a expresse, e é por isso que o problema desce para o código.

A afirmação tem demonstração empírica neste documento. Na janela de 180 dias, as denúncias de assédio em São Paulo formam **dois focos distintos**, de 22 de março a 15 de abril e de 8 de maio a 9 de junho, separados por um intervalo de silêncio. Um `GROUP BY cidade, tipo` devolveria um único grupo de 12 denúncias e apagaria a informação mais útil da tabela: que houve dois episódios, e que o primeiro terminou.

5.6 As quatro estruturas, e o que cada uma resolve

A implementação está em `service/ReincidenciaService.java`, com 397 linhas. A ordem das etapas é a ordem das estruturas: a árvore recorta, a ordenação organiza, o grafo liga e a busca em largura agrupa.

5.6.1 Árvore balanceada: o recorte temporal

```
public static TreeMap<LocalDateTime, List<No>> indexarPorData(List<Denuncia> lista)
public static List<No> recortarJanela(TreeMap<...> arvore,
                                       LocalDateTime inicio, LocalDateTime fim)
```

As denúncias são indexadas por instante de criação num `TreeMap`, que o Java implementa como árvore rubro-negra, com inserção e busca em tempo logarítmico (ORACLE, 2023d).

Por que uma árvore, e não a lista que o banco já devolve ordenada. Uma lista ordenada não responde *o que aconteceu entre estas duas datas* sem percorrer. O `subMap` da árvore devolve uma **visão** da faixa, sem copiar o que ficou de fora: em tempo logarítmico ela localiza a borda da janela e a partir dali percorre apenas os elementos que interessam. A alternativa, filtrar a lista inteira, custaria tempo linear a cada vez que alguém trocasse o período na tela, e o período muda o tempo todo: a interface oferece quatro janelas.

Por que o valor é uma lista. Duas denúncias podem ter o mesmo instante de criação. Guardar um registro por chave faria a segunda desaparecer, e perder uma denúncia por colisão de chave é perder um caso. Há teste automatizado para isso.

5.6.2 Ordenação: agrupar sem agrupar

```
public static void ordenar(List<No> nos) {
    nos.sort(Comparator.comparing(No::chave).thenComparing(No::data));
}
```

A ordenação por chave de grupo e depois por data **não é cosmética**. É ela que transforma a montagem do grafo de quadrática em linear: depois de ordenar, todas as denúncias de um mesmo grupo ficam contíguas e em ordem cronológica, e basta comparar cada uma com a seguinte.

O algoritmo é o `sort` da biblioteca padrão, que para objetos usa uma variante de *merge sort* estável com custo garantido de O(n log n) (ORACLE, 2023e). A escolha de não implementar uma ordenação própria é deliberada: a biblioteca padrão é mais rápida, mais testada e mais legível do que qualquer implementação escrita para um trabalho acadêmico, e trocá-la seria substituir código correto por código demonstrativo.

A **estabilidade** do algoritmo importa aqui: com chave composta, um algoritmo instável poderia reordenar registros de mesma data, e o resultado deixaria de ser determinístico entre execuções.

5.6.3 Busca binária: localizar o bloco de um grupo

```
public static int primeiroDoGrupo(List<No> ordenados, String chave) {
    int lo = 0, hi = ordenados.size() - 1, achado = -1;
    while (lo <= hi) {
        int meio = (lo + hi) >>> 1;
        int cmp = ordenados.get(meio).chave().compareTo(chave);
        if (cmp == 0)     { achado = meio; hi = meio - 1; }
        else if (cmp < 0) { lo = meio + 1; }
        else              { hi = meio - 1; }
    }
    return achado;
}
```

Busca binária clássica, com **duas diferenças que merecem explicação**.

Ao encontrar a chave, ela não para. Continua para a esquerda procurando se há outra igual antes, porque a pergunta é *onde o bloco começa* e não *onde há uma ocorrência*. Parar no primeiro acerto devolveria um ponto qualquer do meio do bloco, e o chamador perderia os registros anteriores sem perceber. Esta é a variante conhecida como *lower bound*, e a distinção entre ela e a busca binária ingênua é exatamente o tipo de detalhe que Knuth (1998) observa ser fonte recorrente de erro.

O ponto médio usa `>>>` e não `/2` sobre a soma. `(lo + hi) / 2` transborda quando a soma excede o maior inteiro positivo, defeito que existiu durante anos na própria biblioteca padrão do Java. O deslocamento sem sinal trata o valor transbordado como não negativo e produz o resultado correto.

Onde ela é usada. Não no laço principal, que percorre o vetor uma vez. Ela serve à pergunta pontual *este caso novo entra em algum foco existente?*, feita pela ficha de um caso específico. Percorrer a janela inteira para responder custaria tempo linear por consulta aberta.

Há um teste que compara, para doze grupos distintos, o resultado da busca binária com o de uma varredura linear. **O ganho da busca binária é de tempo, nunca de resultado**; se ela divergir da varredura, o ganho não vale nada.

5.6.4 Grafo e busca em largura: o agrupamento

```
public static List<List<Integer>> montarGrafo(List<No> ordenados, int janelaDias)
public static List<List<Integer>> componentesConexos(List<List<Integer>> adj)
```

O grafo é não dirigido, representado por lista de adjacência. Os vértices são as denúncias da janela; as arestas ligam denúncias compatíveis e próximas no tempo.

Os componentes conexos são obtidos por **busca em largura**, com fila.

Por que largura e não profundidade. As duas encontram os mesmos componentes e têm o mesmo custo assintótico. A diferença é prática: um foco longo é uma corrente de centenas de vértices, e a versão recursiva da busca em profundidade abre um quadro de pilha por nível. Uma cadeia de dez mil relatos estouraria a pilha. A busca em largura usa uma fila alocada no *heap*, implementada com `ArrayDeque` (ORACLE, 2023c), e não tem esse teto.

Isso não é hipótese: **há um teste automatizado que percorre uma cadeia de dez mil denúncias**, e ele existe para que a decisão continue valendo quando alguém, no futuro, considerar trocar a fila por recursão.

5.7 O detalhe que muda o custo

Este é o ponto fino do algoritmo, e ele merece seção própria porque é onde a solução ingênua e a adotada divergem em ordem de grandeza.

A solução ingênua liga todos os pares de denúncias compatíveis e próximas. Para um grupo com k denúncias, isso são até k(k−1)/2 comparações e outras tantas arestas. Num grupo de 200 relatos, quase vinte mil arestas para descrever **um único foco**.

A solução adotada liga apenas denúncias **consecutivas** do mesmo grupo, no vetor já ordenado:

```
for (int i = 0; i + 1 < n; i++) {
    No a = ordenados.get(i), b = ordenados.get(i + 1);
    if (!a.chave().equals(b.chave())) continue;
    if (ChronoUnit.DAYS.between(a.data(), b.data()) <= janelaDias) {
        adj.get(i).add(i + 1);
        adj.get(i + 1).add(i);
    }
}
```

Por que isso basta. Se A liga em B e B liga em C, os três caem no mesmo componente mesmo sem aresta entre A e C. A conectividade se encarrega da transitividade, que é precisamente a propriedade que define *foco* neste problema. O grafo fica com no máximo k−1 arestas.

Há teste automatizado verificando a propriedade: com 200 vértices no mesmo grupo, o grafo tem exatamente 199 arestas e um único componente.

O que se perde. Nada, para a pergunta feita. Perde-se a capacidade de responder *quais denúncias estão diretamente ligadas a esta*, que exigiria as arestas todas. Essa pergunta não é feita pelo sistema, e acrescentá-la exigiria rever a decisão.

5.8 Análise de complexidade

A tabela abaixo decompõe o custo. A notação segue Cormen et al. (2022), com **n** como o total de denúncias na base e **k** como o número delas dentro da janela analisada.

Etapa	Complexidade	Justificativa
Indexar por data	O(n log n)	n inserções em árvore rubro-negra, cada uma O(log n)
Recortar a janela	O(log n + k)	`subMap` localiza a borda em O(log n) e percorre k
Ordenar a janela	O(k log k)	*merge sort* estável da biblioteca padrão
Localizar um grupo	O(log k)	busca binária no vetor ordenado
Montar o grafo	O(k)	uma comparação por par consecutivo
Componentes conexos	O(V + E) = O(k)	E ≤ k−1 pela construção da seção 5.7
Montar o resultado	O(k log k)	ordenação dos componentes por data
Total por consulta	**O(k log k)**	a ordenação domina

Espaço. O índice ocupa O(n); a janela, o vetor ordenado e as listas de adjacência ocupam O(k). Como E ≤ k−1, a lista de adjacência não é a estrutura dominante, o que não seria verdade na solução ingênua, onde ela seria $O(k^2)$.

Comparação com a alternativa ingênua.

	Solução ingênua	Solução adotada
Arestas	até k(k−1)/2	no máximo k−1
Montagem	$O(k^2)$	O(k)

	Solução ingênua	Solução adotada
do grafo		
Percurso	$O(k^2)$	$O(k)$
Total	$O(k^2)$	$O(k \log k)$
Memória do grafo	$O(k^2)$	$O(k)$

Para k igual a 160, que é o tamanho da janela de 90 dias na base atual, a diferença é entre cerca de 12,7 mil comparações e 159. Para k igual a 10.000, entre quase 50 milhões e 10 mil.

Uma honestidade sobre estes números. Em k igual a 160, a diferença é irrelevante na prática: as duas soluções respondem em milissegundos. A justificativa da escolha não é o desempenho de hoje, é o fato de que a solução quadrática **deixa de funcionar** num volume que uma operação real alcança, e trocar a estrutura depois custa mais do que acertá-la agora.

5.9 O achado: foco não é o mesmo que volume

Esta seção registra uma limitação conceitual que só ficou visível depois de a tela estar pronta e rodando sobre a base real. Ela está aqui porque é o achado mais interessante da Parte 4.

5.9.1 O que apareceu na tela

Com a implementação descrita até aqui, a janela de 90 dias produziu nove agrupamentos. O maior deles: **Barueri, violência, 49 denúncias, 41 dias**.

O algoritmo estava correto. Aquelas 49 denúncias formam mesmo uma cadeia contínua: nenhuma delas ficou mais de 14 dias distante da seguinte.

O problema é de significado, e não de código. **Aquele agrupamento reunia 100% de tudo o que Barueri produziu naquele tipo na janela.** Ele não destacava nada: ele *era* o volume do município. Uma equipe que tratasse aquilo como foco montaria uma operação contra a operação inteira do lugar.

Pior: era o problema do `GROUP BY` voltando pela porta dos fundos. Quando o dado é denso o suficiente, a cadeia nunca se rompe e o componente engole o grupo todo, produzindo exatamente o balde que a seção 5.5 argumenta ser inútil.

5.9.2 A correção: nomear o que o algoritmo encontrou

A correção não foi mexer no grafo. Foi reconhecer que o algoritmo encontra **duas coisas diferentes** e que elas pedem ações diferentes.

Foco é um surto que começa e termina dentro da janela. Pede operação pontual.

Fluxo contínuo é uma cadeia que nunca se interrompe porque a demanda daquele lugar é constante. Pede equipe permanente.

A distinção é feita por cobertura:

```
public static boolean ehFluxoContinuo(int tamanhoDoComponente, int totalNoGrupo) {
    if (totalNoGrupo < MINIMO_PARA_SATURACAO) return false;   // 10
    return (double) tamanhoDoComponente / totalNoGrupo >= COBERTURA_DE_SATURACAO; //
0,70
}
```

Por que uma fração e não um número absoluto. Um limite do tipo *acima de 25 é fluxo* erraria nas duas pontas: 30 denúncias num grupo de 100 destacam alguma coisa, e 30 num grupo de 40 não destacam. Há teste automatizado com exatamente esses dois casos.

Por que o piso de 10. Num grupo de 4 denúncias, o componente quase sempre cobre quase tudo. Aplicar o corte ali marcaria como *volume normal* aquilo que é apenas grupo pequeno. O efeito do piso é visível nos resultados: *Barueri, discriminação* e *Barueri, assédio* têm cobertura de 100% e permanecem classificados como focos, porque seus grupos têm 7 e 4 denúncias.

5.9.3 Os resultados sobre a base real

Base de 190 denúncias, janela de 90 dias, ancorada na denúncia mais recente:

Classificação	Município	Tipo	Qtd	Período	Duração	Cobertura
Foco	Barueri	discriminação	7	04/04 a 13/05	39 d	100,0%
Foco	São Paulo	assédio	5	08/05 a 09/06	31 d	55,6%
Foco	Barueri	assédio	4	01/05 a 11/05	10 d	100,0%
Foco	Barueri	outros	4	06/05 a 09/05	3 d	80,0%
Foco	Rio de Janeiro	assédio	3	31/03 a 21/04	20 d	100,0%
Foco	São Paulo	assédio	3	31/03 a 15/04	14 d	33,3%

Classificação	Município	Tipo	Qtd	Período	Duração	Cobertura
Foco	Belo Horizonte	discriminação	3	09/04 a 27/04	18 d	75,0%
Fluxo contínuo	Barueri	violência	49	03/04 a 14/05	41 d	100,0%
Fluxo contínuo	São Paulo	violência	11	04/04 a 01/06	57 d	84,6%

São 9 agrupamentos, 89 denúncias agrupadas, **7 focos e 2 fluxos contínuos**. Depois da correção, o maior foco passou de 49 para 7, e os dois agrupamentos que eram o volume do próprio lugar foram para o fim da lista, com o rótulo que os descreve.

Duas linhas merecem leitura conjunta, porque elas exibem a regra do piso em funcionamento. *Barueri*, *assédio* tem cobertura de 100% e continua sendo foco, porque o grupo tem 4 denúncias; *Barueri*, *violência* tem a mesma cobertura de 100% e vira fluxo contínuo, porque o grupo tem 49. A cobertura sozinha não decide: ela só passa a significar alguma coisa quando há grupo suficiente para que cobrir tudo seja informativo.

5.9.4 Dois comportamentos que a base revela

A janela de 30 dias devolve zero agrupamentos. A denúncia mais recente da base é de 28 de junho e a carga de dispositivos termina em 15 de maio, de modo que os últimos 30 dias contêm 27 denúncias esparsas, sem cadeia de três. Isso **não é defeito**: é a resposta correta, e ela informa. Uma tela que devolvesse algo em toda janela estaria inventando agrupamento.

A janela de 180 dias separa o que a de 90 unia. As denúncias de assédio em São Paulo aparecem como **dois focos** naquela janela, de 22/03 a 15/04 e de 08/05 a 09/06, e a janela de 90 dias, por começar depois de 31 de março, só alcança o segundo episódio inteiro. É a demonstração empírica do argumento da seção 5.5: a cadeia se rompeu, o primeiro episódio terminou e outro começou. Nenhum agrupamento por chave expressaria isso.

5.10 Por que a leitura de focos é auditada

A rota `GET /api/reincidencia/focos` é anotada com `@Auditavel(acao = CONSULTA_SENSIVEL)`, o mesmo tratamento dado à leitura do relato de uma denúncia.

A justificativa é menos evidente do que a das outras rotas sensíveis, e por isso precisa ser escrita. **A tela não mostra nenhum dado pessoal.** Ela mostra município, tipo, datas, contagens e protocolos.

O que ela mostra é uma informação que **não existe em nenhuma denúncia isolada**: que vários relatos anônimos descrevem a mesma situação em curso, num lugar determinado. Isso é uma afirmação sobre um território e sobre as pessoas que vivem nele, e é o tipo de informação agregada que Sweeney (2002)

e Rocher, Hendrickx e De Montjoye (2019) mostram poder estreitar o conjunto de pessoas compatíveis com um relato, ainda que cada relato isolado seja anônimo.

Um agrupamento diz mais do que a soma dos itens agrupados. Por isso ele fica no mesmo regime de auditoria.

5.11 Limitações declaradas da Parte 4

A granularidade é o município, e município é grosso. Duas denúncias em pontas opostas de uma cidade grande são ligadas pelo critério atual. A saída correta é bairro ou coordenada, e o esquema atual não carrega nenhum dos dois para denúncia digitada à mão: `DENUNCIA` tem `endereco` como texto livre. Esta é a limitação mais séria da solução.

A janela de 14 dias é uma escolha, e não um resultado. Ela foi fixada por raciocínio: encurtar fragmenta focos reais, alargar cola situações não relacionadas. Não há, nesta base, dado suficiente para calibrá-la. O parâmetro está isolado e nomeado em `JANELA_ELO_DIAS` exatamente para poder ser ajustado quando houver operação real.

O limiar de 70% tem a mesma natureza. Ele separa foco de fluxo contínuo por um critério defensável e não medido.

A análise roda inteira a cada consulta. Não há resultado armazenado. Para a base atual isso é irrelevante, e para uma base de centenas de milhares de denúncias exigiria cálculo incremental ou materialização periódica.

Os textos das denúncias não são usados. O agrupamento usa apenas metadados. Isso é uma limitação de precisão adotada de propósito, pelas razões registradas na seção 3.7: usar o relato exigiria processá-lo, e ele é o dado mais sensível da base.

A consulta 3 não tem volume para concluir. Trinta e quatro intervalos observados permitem montar o instrumento, não calibrar o parâmetro.

5.12 Verificação

O que é verificado	Como
As seis consultas devolvem os resultados publicados	Execução registrada em `database/05_resultados_fase6.txt`
A carga é reproduzível	`python gerar_carga_fase6.py`, semente 561601
Os focos publicados conferem com a base	Reprodução independente do algoritmo sobre `data.sql`
As estruturas fazem o que o documento afirma	19 testes em `ReincidenciaServiceTest`

O que é verificado	Como
A busca binária concorda com a varredura	Teste de equivalência em 12 grupos
O grafo tem no máximo k−1 arestas	Teste com 200 vértices, 199 arestas
A busca em largura não estoura a pilha	Teste com cadeia de 10.000 vértices
A transitividade funciona	Teste com pontas a 20 dias, ligadas por intermediário
A cadeia interrompida separa focos	Teste com 55 dias de silêncio
O limiar de saturação é relativo	Teste com 30/100 e 30/40

5.13 Síntese da Parte 4

Exigência do enunciado	Onde foi atendida
Consultas com agrupamentos, totais ou métricas	Seções 5.2.1, 5.2.2, 5.2.5 e 5.2.6
Ao menos uma subconsulta ou função avançada	`ROLLUP` em 5.2.1; `RANK` e `SUM OVER` em 5.2.2; `LAG` em 5.2.3; subconsulta em 5.2.4
Apresentar os resultados obtidos	Seção 5.2 e Anexo C
Explicar as informações extraídas	Em cada consulta, no bloco *o que isto informa*
Situação de busca, organização, ordenação ou relacionamento	Seção 5.4
Árvores	Seção 5.6.1
Busca	Seção 5.6.3
Ordenação	Seção 5.6.2
Grafos	Seção 5.6.4
Análise de complexidade com Big O	Seção 5.8
Explicar o problema	Seções 5.4 e 5.5
Justificar a solução escolhida	Seções 5.5, 5.6 e 5.7
Demonstrar a aplicação no projeto	Seção 5.9.3 e tela **Focos de reincidência**

Conclusão

6.1 O que foi entregue

A Fase 6 acrescentou ao Protege+ duas capacidades que o sistema não tinha e corrigiu uma lacuna estrutural na forma como ele era verificado.

A primeira capacidade é **receber denúncia sem que ninguém digite**. Três origens de dispositivo, um botão de pânico, uma tornozeleira com perímetro e um totem público, escrevem num barramento único de eventos. O servidor decide o que vira caso, e a classificação de urgência continua saindo do texto, lida pelo mesmo componente que lê um relato digitado por uma pessoa.

A segunda é **relacionar denúncias entre si**. Uma análise por componentes conexos identifica cadeias de relatos anônimos que descrevem a mesma situação em curso, distinguindo um surto que começa e termina da demanda constante de um lugar.

A lacuna corrigida é de método: a suíte de testes passou a falhar quando a aplicação não sobe, o que antes não acontecia e permitiu que o projeto estivesse, por algum tempo, num estado em que 116 testes passavam e a aplicação não iniciava.

6.2 O que não foi feito, e por quê

Esta seção é tão importante quanto a anterior, e ela existe porque um relatório que só descreve acertos não permite avaliar critério.

Não há hardware. Os três dispositivos são simulados por uma página que fala o mesmo protocolo. Isso prova que o lado do servidor está completo e não prova nada sobre consumo de energia, alcance de rádio ou comportamento em rede instável.

Não há fila de mensagens. O transporte é HTTP síncrono, e um aparelho sem rede perde o evento. O desenho correto para operação real usa um intermediário com *buffer*. Ele não foi construído porque acrescentaria infraestrutura que não pode ser demonstrada num trabalho acadêmico sem virar encenação, e porque o contrato de entrada permanece o mesmo quando ele for acrescentado.

Não há detecção de silêncio. O sistema reage ao que chega. Uma tornozeleira que para de transmitir, que é exatamente o que aconteceria se ela fosse rompida, não gera alerta. Esta é a falha mais séria da origem B, e ela é declarada porque escondê-la daria ao leitor uma impressão de completude que a implementação não sustenta.

Não há integração com órgão de segurança pública. Um alerta de descumprimento de medida protetiva abre um caso no painel do Protege+ e não aciona ninguém fora dele. Em operação real, esse é o passo que dá efeito à funcionalidade, e ele depende de acordo institucional, não de código.

A granularidade do agrupamento é o município. Duas denúncias em pontas opostas de uma cidade grande são ligadas pelo critério atual. A correção exige bairro ou coordenada, e o esquema não carrega nenhum dos dois para denúncia digitada à mão.

Os parâmetros não foram calibrados com dado. A janela de repique de 120 segundos, o intervalo de 600 segundos entre alertas, a janela de elo de 14 dias e o limiar de saturação de 70% são escolhas justificadas por raciocínio e não medidas em operação. A consulta da seção 5.2.3 é o instrumento que permitirá calibrar a primeira delas; as demais ainda não têm instrumento.

Os textos gerados pelo servidor são gravados sem acentuação. A convenção vem das fases anteriores e alcança quase todo o código de produção: apenas dois dos 79 arquivos Java contêm cadeias acentuadas. O efeito é visível para quem atende, porque o relato montado a partir de um acionamento de dispositivo aparece no painel como *Pedido de socorro enviado agora por botao de panico fisico*.

A correção é simples de descrever e arriscada de aplicar nesta altura: acentuar as cadeias exige atualizar as asserções de teste que comparam texto exato, ajustar o gerador de carga e regerar os 181 registros de demonstração, e uma correção que não pode ser inteiramente verificada antes da entrega vale menos do que a declaração dela. Fica registrada como dívida, com a mesma régua aplicada ao item herdado descrito no parágrafo seguinte.

O defeito declarado na Fase 5 permanece. O item 11 do Anexo D daquele documento, sobre o canal identificado coletar nome e contato que a API não recebe, não foi corrigido nesta fase. Ele segue declarado, e a razão de não ter sido tratado agora é que o escopo desta fase foi definido pelo enunciado, que trata de IA, IoT, SQL e estruturas de dados.

6.3 Próximos passos

Prioridade	Passo	Por quê
1	Detecção de silêncio de equipamento	É a falha que mais compromete a origem B
2	Certificado por dispositivo, com rotação	Substitui a chave compartilhada
3	Bairro ou coordenada na denúncia	Corrige a granularidade do agrupamento
4	Intermediário com fila para os dispositivos	Torna o transporte tolerante a falha de rede
5	Cadastro do parque de botões e tornozeleiras	Hoje só o totem tem cadastro de instalação
6	Uso do dado de bateria	Já é coletado e não dispara nada

Prioridade	Passo	Por quê
7	Integração com órgão de segurança pública	Dá efeito ao alerta fora do painel
8	Correção do defeito 11 herdado da Fase 5	Dívida declarada há uma fase
9	Acentuação das cadeias geradas pelo servidor	Texto que o servidor público lê

6.4 Uma observação final sobre método

Três dos achados mais úteis desta fase não vieram de planejamento, e sim de execução.

O primeiro foi descobrir que 116 testes passavam e a aplicação não subia. Ele não seria encontrado por leitura de código nem por revisão de arquitetura: apareceu porque alguém tentou executar.

O segundo foi perceber que o gerador de carga sorteava intervalos dentro da janela que a consulta deveria auditar. Ele apareceu ao escrever a explicação do resultado, quando a conclusão ficou boa demais para ser verdade.

O terceiro foi ver, na tela pronta, que o maior foco reunia 100% do que aquele município produziu naquele tipo. O algoritmo estava certo, e o conceito estava incompleto.

Os três têm a mesma assinatura: **eles apareceram no momento em que o trabalho saiu do abstrato**. Isso é um argumento a favor de construir a tela, rodar a consulta e executar a aplicação antes de declarar qualquer coisa pronta, e é a razão de a definição de pronto da seção 3.8 exigir as três coisas.

Referências

As fontes abaixo sustentam as decisões de projeto defendidas neste documento. Legislação e norma técnica vêm sempre da fonte primária. Onde a fonte primária não pôde ser lida integralmente, isso está dito na nota ao final desta seção.

Legislação e normas do poder público brasileiro

BRASIL. **Lei nº 11.340, de 7 de agosto de 2006.** Cria mecanismos para coibir a violência doméstica e familiar contra a mulher. Brasília: Presidência da República, 2006. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2004-2006/2006/lei/l11340.htm

BRASIL. **Lei nº 12.258, de 15 de junho de 2010.** Altera o Código Penal e a Lei de Execução Penal para prever a possibilidade de utilização de equipamento de vigilância indireta pelo condenado. Brasília: Presidência da República, 2010. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2010/lei/l12258.htm

BRASIL. **Decreto nº 7.627, de 24 de novembro de 2011.** Regulamenta a monitoração eletrônica de pessoas. Brasília: Presidência da República, 2011. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2011-2014/2011/decreto/d7627.htm

BRASIL. **Lei nº 13.460, de 26 de junho de 2017.** Dispõe sobre participação, proteção e defesa dos direitos do usuário dos serviços públicos da administração pública. Brasília: Presidência da República, 2017.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018.** Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília: Presidência da República, 2018a. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm

BRASIL. **Lei nº 13.608, de 10 de janeiro de 2018.** Dispõe sobre o serviço telefônico de recebimento de denúncias. Brasília: Presidência da República, 2018b.

BRASIL. **Decreto nº 10.153, de 3 de dezembro de 2019.** Dispõe sobre as salvaguardas de proteção à identidade dos denunciantes de ilícitos e de irregularidades praticados contra a administração pública federal direta e indireta. Brasília: Presidência da República, 2019.

AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS (ANPD). **Guia Orientativo sobre Segurança da Informação para Agentes de Tratamento de Pequeno Porte.** Versão 1.0. Brasília: ANPD, 2021.

Algoritmos e estruturas de dados

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Introduction to Algorithms.** 4. ed. Cambridge: MIT Press, 2022. Fundamenta a notação assintótica, a busca em largura e a definição de componente conexo.

HOPCROFT, John; TARJAN, Robert. **Algorithm 447: efficient algorithms for graph manipulation.** Communications of the ACM, v. 16, n. 6, p. 372-378, 1973. Referência clássica dos algoritmos lineares de componentes conexos.

KNUTH, Donald E. **The Art of Computer Programming, Volume 3: Sorting and Searching.** 2. ed. Reading: Addison-Wesley, 1998. Fundamenta a distinção entre a busca binária ingênua e a variante que localiza o primeiro elemento de um bloco.

SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms.** 4. ed. Upper Saddle River: Addison-Wesley, 2011. Referência de apoio sobre árvores de busca balanceadas e representação de grafos por lista de adjacência.

SINNOTT, Roger W. **Virtues of the Haversine.** Sky and Telescope, v. 68, n. 2, p. 158, 1984. Fonte da fórmula de Haversine adotada no cálculo de distância geodésica.

Engenharia de software, requisitos e testes

BECK, Kent. **Test-Driven Development: By Example.** Boston: Addison-Wesley, 2002.

COHN, Mike. **User Stories Applied: For Agile Software Development.** Boston: Addison-Wesley, 2004. Fundamenta o formato das user stories da Parte 2.

FOWLER, Martin. **Patterns of Enterprise Application Architecture.** Boston: Addison-Wesley, 2002. Fonte do padrão *Single Table Inheritance*, adotado no barramento de eventos, e da crítica clássica a ele.

FOWLER, Martin. **TestPyramid.** Bliki, [s.d.]. Disponível em: https://martinfowler.com/bliki/TestPyramid.html

PENG, Roger D. **Reproducible Research in Computational Science.** Science, v. 334, n. 6060, p. 1226-1227, 2011. Fundamenta a exigência de carga determinística com semente declarada.

WIEGERS, Karl; BEATTY, Joy. **Software Requirements.** 3. ed. Redmond: Microsoft Press, 2013. Fundamenta o critério de que um critério de aceitação precisa ser verificável de forma binária.

Segurança da informação e privacidade

KOCHER, Paul C. **Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems.** In: ADVANCES IN CRYPTOLOGY, CRYPTO ’96. Berlim: Springer, 1996. p. 104-113. Fundamenta a comparação de chave em tempo constante.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION. **ISO/IEC 27002:2022:** information security, cybersecurity and privacy protection: information security controls. Genebra: ISO, 2022. Controle 8.15, sobre registro de eventos.

OPEN WORLDWIDE APPLICATION SECURITY PROJECT (OWASP). **OWASP API Security Top 10: 2023.** 2023. Disponível em: https://owasp.org/API-Security/editions/2023/en/0x11-t10/

OPEN WORLDWIDE APPLICATION SECURITY PROJECT (OWASP). **Logging Cheat Sheet.** OWASP Cheat Sheet Series, [s.d.]. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

ROCHER, Luc; HENDRICKX, Julien M.; DE MONTJOYE, Yves-Alexandre. **Estimating the success of re-identifications in incomplete datasets using generative models.** Nature Communications, v. 10, art. 3069, 2019. Fundamenta a recusa da proposta de identificação de reincidência por pessoa.

SWEENEY, Latanya. **k-Anonymity: A Model for Protecting Privacy.** International Journal on Uncertainty, Fuzziness and Knowledge-based Systems, v. 10, n. 5, p. 557-570, 2002. Fundamenta o tratamento do agrupamento como informação sensível.

Documentação das tecnologias utilizadas

MOZILLA. **Cross-Origin Resource Sharing (CORS).** MDN Web Docs, [s.d.]. Disponível em: https://developer.mozilla.org/docs/Web/HTTP/CORS Fundamenta o comportamento de verificação prévia com cabeçalho personalizado.

ORACLE CORPORATION. **Class TreeMap.** Java SE 21 API Specification. Redwood Shores: Oracle, 2023d. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/TreeMap.html

ORACLE CORPORATION. **Interface List, método sort.** Java SE 21 API Specification. Redwood Shores: Oracle, 2023e. Documenta a garantia de estabilidade e o custo O(n log n) da ordenação da biblioteca padrão.

ORACLE CORPORATION. **Class ArrayDeque.** Java SE 21 API Specification. Redwood Shores: Oracle, 2023c.

ORACLE CORPORATION. **PERCENTILE_CONT, ROLLUP e funções analíticas.** Oracle Database SQL Language Reference. Redwood Shores: Oracle, [s.d.]. Disponível em: https://docs.oracle.com/en/database/oracle/oracle-database/21/sqlrf/

RED HAT. **Hibernate ORM User Guide.** Seção sobre mapeamento de tipos básicos e atributos de coluna. [s.d.]. Fundamenta a recusa de escala sobre tipo de ponto flutuante.

VMWARE. **Spring Framework Reference: CORS Configuration.** [s.d.]. Disponível em: https://docs.spring.io/spring-framework/reference/web/webmvc-cors.html

Nota sobre o uso das fontes

Três ressalvas de honestidade intelectual fecham esta seção.

Duas fontes entram como apoio e não são citadas no corpo. Sedgewick e Wayne (2011) e a documentação de `PERCENTILE_CONT` do Oracle sustentaram a redação sem que nenhuma afirmação específica dependa delas; estão listadas por honestidade quanto ao que foi consultado, e não para engrossar a bibliografia.

Nem toda afirmação deste documento precisa de fonte externa, e forçar uma seria pior do que não ter. Os números das Partes 1 e 4 descrevem a base de demonstração deste projeto e não têm outra fonte além do próprio repositório; eles são verificáveis pela reexecução do gerador, e não por citação.

Onde a fonte primária não pôde ser lida integralmente, isso está dito. O texto do controle 8.15 da ISO/IEC 27002:2022 é norma paga e foi citado pelo número e pelo tema, sem reprodução do conteúdo.

As decisões de projeto deste documento não são derivadas das fontes, e sim sustentadas por elas. A escolha de manter o perímetro no servidor, por exemplo, não está escrita em norma alguma: ela decorre do raciocínio exposto na seção 4.6.2, e a LGPD é citada porque o princípio da minimização converge com ele, não porque o determine.

Anexos

Anexo A: rotas acrescentadas nesta fase

O sistema passou de 31 para 38 endpoints. Os sete novos estão abaixo. Todos estão documentados em OpenAPI e visíveis em `http://localhost:8080/swagger-ui.html`.

Método	Rota	Autenticação	Código de sucesso	Auditada
POST	`/api/iot/acionamento`	`X-Device-Key`	201	Sim, quando abre caso
POST	`/api/iot/telemetria`	`X-Device-Key`	202	Sim, quando abre caso
POST	`/api/iot/totem`	`X-Device-Key`	201, ou 200 em reenvio	Sim, quando abre caso
GET	`/api/iot/eventos`	JWT	200	Sim, consulta sensível
GET	`/api/iot/resumo`	JWT	200	Não
GET	`/api/reincidencia/focos`	JWT	200	Sim, consulta sensível
GET	`/api/reincidencia/parametros`	JWT	200	Não

A.1 Códigos de recusa e o que cada um significa

Código	Quando	Por que este e não outro
401	Chave de dispositivo ausente ou inválida	O emissor não se autenticou
422	Totem não habilitado, ou relato insuficiente	O corpo está bem formado; o conteúdo é que não serve. A distinção importa ao próprio totem, que precisa saber se reapresenta a tela de digitação ou avisa que está fora de serviço

Código	Quando	Por que este e não outro
400	Corpo malformado ou campo obrigatório ausente	Falha de contrato, tratada pela validação declarativa
403	Perfil sem acesso à rota de leitura	Registrado na trilha, conforme US33

A.2 Quando a trilha registra, e quando não registra

As três rotas de dispositivo gravam na trilha de auditoria **apenas quando abrem um caso**. Um evento reconhecido como repique, ou descartado por validação, é gravado na tabela do barramento e não gera entrada na trilha.

A distinção é deliberada e vale ser declarada, porque a leitura apressada da tabela acima sugeriria o contrário. A trilha encadeada da US25 registra ações sobre **dados de denúncia**; um repique não altera dado de denúncia algum, ele apenas anota que um aparelho enviou de novo. Poluir a trilha com telemetria descartada reduziria a densidade de informação do registro que existe para apurar acesso indevido, contrariando a orientação de registrar os eventos que importam em vez de registrar tudo (OWASP, [s.d.]).

O rastro do que foi descartado não se perde: ele está na tabela `EVENTO_IOT`, com a situação e o motivo, e é exatamente o que a tela **Dispositivos de campo** exibe.

A.3 O que as rotas de dispositivo não fazem

Não existe rota que permita a um aparelho **ler**, **alterar** ou **excluir**. As três rotas de dispositivo apenas criam. Essa restrição é o que torna aceitável a autenticação por chave compartilhada, conforme discutido na seção 4.10.1, e está declarada em `SecurityConfig` com comentário explicando o motivo.

Anexo B: objetos do banco acrescentados

`database/04_ddl_fase6.sql` é aditivo: ele não altera nenhum objeto das fases anteriores.

Tabela	PK	UNIQUE	CHECK	Índices	Comentários
`EVENTO_IOT`	1	0	6	2	6
`ZONA_EXCLUSAO`	1	1	5	0	6
`PONTO_TOTEM`	1	1	4	0	5
Total	**3**	**2**	**15**	**2**	**17**

B.1 Restrições que expressam regra de negócio

Três `CHECK` merecem registro porque não são validação de tipo, e sim regra de domínio replicada no banco para o caso de a validação da aplicação ser algum dia contornada.

`ck_evento_iot_coord` exige que latitude e longitude sejam ambas nulas ou ambas preenchidas. Coordenada só faz sentido completa: guardar apenas a latitude seria guardar um dado que não responde pergunta nenhuma.

`zona_exclusao_uk_torn` garante que um equipamento vigie um perímetro por vez. Duas zonas ativas para a mesma tornozeleira gerariam dois alertas para o mesmo passo, e uma central que recebe alerta em dobro aprende a desconfiar do alerta. A restrição protege a credibilidade do aviso, não a limpeza da tabela.

`ck_zona_raio` limita o raio entre 50 e 20.000 metros. O piso evita perímetro menor que a imprecisão do próprio sinal; o teto evita que um erro de digitação transforme um bairro numa área de exclusão.

Anexo C: resultados completos das consultas

Os resultados das seis consultas estão reproduzidos na seção 5.2 e registrados na íntegra em `database/05_resultados_fase6.txt`, junto da leitura de cada um.

Para reproduzir:

```
1. cd backend && build.cmd spring-boot:run
2. abrir http://localhost:8080/h2-console
   JDBC URL: jdbc:h2:file:./data/protege
   usuário: sa      senha: (vazia)
3. colar o conteúdo de database/05_consultas_fase6.sql
```

As duas divergências de sintaxe entre Oracle e H2 estão anotadas nos comentários do próprio script, nas consultas 3 e 4.

Para reproduzir a carga:

```
cd database && python gerar_carga_fase6.py > carga.sql
```

A saída é idêntica em qualquer máquina, porque a semente do gerador é 561601.

Anexo D: defeitos encontrados nesta fase

Os quinze defeitos abaixo foram encontrados **durante** o desenvolvimento, e não depois da entrega. Eles estão listados porque a forma como um defeito é encontrado diz mais sobre o processo do que a ausência de defeitos diria.

#	Defeito	Como se manifestava	Como foi encontrado
1	`String.format` sem `Locale.ROOT`	Coordenada com vírgula em máquina configurada em português. Não	Teste que falhou

#	Defeito	Como se manifestava	Como foi encontrado
		apareceria em servidor em inglês	
2	`precision` e `scale` sobre `Double`	A aplicação não subia, com 116 testes passando	Tentativa de executar
3	Padrão de busca do teste casava com o número do processo	Teste reprovava o dado que ele próprio exigia	Falha do teste, com diagnóstico invertido
4	`X-Device-Key` ausente na política de CORS	Verificação prévia recusada; o `POST` nunca existia	Análise preventiva antes do primeiro uso
5	Botões de direção do simulador invertidos	Impossível entrar no perímetro por aquele caminho	Uso real da tela
6	Passo de longitude usando o divisor de latitude	Deslocamento lateral 9% maior que o anunciado	Revisão do cálculo após o defeito 5
7	Resposta lida pelo nome de campo errado	Tela exibia *caso undefined aberto*	Uso real da tela
8	`COUNT(e.id)` em vez de `COUNT(*)`	Consulta 4 devolvia vazio	Execução da consulta em banco de verificação
9	Gerador sorteava intervalos dentro da janela auditada	A consulta 3 concluiria por construção	Redação da explicação do resultado
10	Carga apontando para protocolos inexistentes	A consulta 5 devolveria vazio	Revisão do modelo antes de gerar
11	Vinte e nove seletores sem par no tema escuro	Metade do painel permanecia claro	Uso real da tela, e auditoria por varredura
12	Função de troca de tema chamada e nunca escrita	Botão não fazia nada, sem erro visível	Uso real da tela

#	Defeito	Como se manifestava	Como foi encontrado
13	Indicador de conexão usando classe CSS extinta	Teste de conexão funcionava e o indicador não mudava	Uso real da tela

14 | Gerador divergia do `VigiaService` num degrau da regra | Relato de tipo *outros* com marcador de imediatismo seria classificado a menos | Auditoria linha a linha do gerador contra o serviço | 15 | Gerador atribuía o `score` em vez de calculá-lo | A carga afirmava sobre o sistema algo que o `ScoreService` não faz | Auditoria do gerador contra o serviço |

D.1 O que este quadro mostra

Seis dos quinze defeitos foram encontrados por **uso real da tela**, e não por teste nem por revisão de código. Dois deles, os de número 12 e 13, são do tipo que não produz erro algum no console: uma função inexistente chamada por `onclick` e uma classe CSS que deixou de existir falham em silêncio.

Três defeitos, os de número 9, 10 e 11, foram encontrados **ao escrever a explicação** do que havia sido feito. Esse é um argumento a favor de documentar enquanto se constrói: a redação obriga a declarar o raciocínio, e um raciocínio declarado é auditável por quem o escreve.

Um defeito, o de número 2, mudou o método de trabalho e gerou a US44.

Os dois últimos, 14 e 15, apareceram numa **auditoria final feita com o documento já escrito**, comparando cada afirmação do texto com o código que deveria sustentá-la. Os dois estavam no gerador de carga, e os dois tinham a mesma assinatura: o programa afirmava reproduzir uma regra do sistema e reproduzia uma aproximação dela. O caso 15 é o mais instrutivo, porque o `score` gerado era *plausível*, coerente com a urgência, e por isso não chamaria atenção de ninguém lendo os dados. Ele só apareceu quando o texto do documento foi confrontado com `ScoreService.java`, linha a linha.

A correção do caso 14 não alterou um único registro da carga, o que foi verificado comparando a saída antes e depois: nenhum relato da base tinha simultaneamente gravidade baixa e marcador de imediatismo. A correção do caso 15 alterou a carga inteira, porque removeu um sorteio da sequência determinística, e por isso todos os números da Parte 4 foram reapurados.

D.2 Um defeito de conceito, e não de código

O caso descrito na seção 5.9 não está na tabela acima porque não é um defeito de implementação. O algoritmo de agrupamento estava correto; a **conclusão** que ele sugeria é que estava errada, porque um agrupamento que reúne todo o grupo não destaca nada.

Ele é registrado aqui porque é o tipo de problema que nenhuma suíte de testes encontraria: não havia asserção falhando, havia uma tela mostrando um número correto que levaria a uma decisão errada.

Anexo E: inventário de arquivos

E.1 Arquivos criados

Arquivo	Linhas	Parte
`backend/.../model/EventoIot.java`	113	3
`backend/.../model/ZonaExclusao.java`	89	3
`backend/.../model/PontoTotem.java`	82	3
`backend/.../repository/EventoIotRepository.java`	26	3
`backend/.../repository/ZonaExclusaoRepository.java`	14	3
`backend/.../repository/PontoTotemRepository.java`	20	3
`backend/.../service/EventoIotService.java`	522	3
`backend/.../service/ReincidenciaService.java`	397	4
`backend/.../controller/IotController.java`	145	3
`backend/.../controller/ReincidenciaController.java`	54	4
`backend/.../dto/` (seis arquivos)	214	3
`backend/src/test/.../EventoIotServiceTest.java`	35 testes	3
`backend/src/test/.../ReincidenciaServiceTest.java`	19 testes	4
`backend/src/test/.../ContextoSobeTest.java`	1 teste	1
`database/04_ddl_fase6.sql`	163	3
`database/05_consultas_fase6.sql`	251	4
`database/05_resultados_fase6.txt`	184	4

Arquivo	Linhas	Parte
`database/gerar_carga_fase6.py`	261	4
`simulador-iot.html`	691	3
`js/dispositivos.js`	140	3
`js/reincidencia.js`	173	4
`css/dispositivos.css`	105	3
`css/reincidencia.css`	112	4
`css/tema-escuro.css`	238	1

E.2 Arquivos alterados

Arquivo	Alteração
`index.html`	Duas telas novas, atalho para o simulador, extração do tema escuro
`js/main.js`	Registro das abas, cores de gráfico por tema, ocultação do chatbot no painel
`js/backend.js`	Quatro métodos de acesso às rotas novas
`js/acesso.js`	Permissões das duas telas novas
`js/estatistica.js`	Cores de gráfico lidas do tema
`js/i18n.js`	Dez chaves novas por idioma, 373 em cada
`css/painel-api.css`	Ajustes de contraste
`backend/.../security/SecurityConfig.java`	Rotas de dispositivo e cabeçalho no CORS
`backend/.../model/EventoIot.java`	Correção do mapeamento de coordenada
`backend/src/main/resources/data.sql`	Carga de demonstração da Fase 6
`backend/src/main/resources/application.properties`	Chave de dispositivo por variável de ambiente

Anexo F: comandos de verificação

F.1 Executar o projeto

```
cd backend
build.cmd clean test          # 136 testes, sem falhas
build.cmd spring-boot:run     # sobe na porta 8080
```

Em outra janela, na pasta do site:

```
python -m http.server 5500
```

O site precisa ser servido por HTTP. Abrir o arquivo por duplo clique faz a página chegar à API com origem nula, e a política de CORS recusa. O motivo está na seção 4.10.2.

F.2 Percorrer a demonstração

Passo	Onde	O que observar
1	`localhost:5500/simulador-iot.html`	Indicador verde, API no ar
2	Botão de pânico, duas vezes seguidas	A segunda vira duplicado, mesmo protocolo
3	Tornozeleira em 750 m, enviar	Sem alerta, fora do perímetro
4	Andar Sul, enviar, andar Sul, enviar	Na terceira, violação com protocolo
5	Enviar mais uma vez	Silêncio, é o intervalo de 600 s
6	Totem TT-001, TT-003 e TT-999	Aceito, recusado, recusado
7	Painel, Dispositivos de campo	Contadores e as três situações
8	Painel, Focos de reincidência	Sete focos e dois fluxos contínuos
9	Abrir *Como isto é calculado*	Tabela de complexidade

F.3 Reproduzir os números deste documento

```
# contagem de testes
cd backend && build.cmd clean test

# contagem de classes Java de produção
find backend/src/main/java -name "*.java" | wc -l

# contagem de endpoints
grep -rhoE "@(Get|Post|Put|Patch|Delete)Mapping" backend/src/main/java | wc -l

# carga determinística
cd database && python gerar_carga_fase6.py | grep -c "INSERT INTO"
```

Anexo G: roteiro do vídeo

Duração alvo: 4 minutos e 40 segundos, dentro do limite de 5 minutos. A comanda pede pitch e demonstração da aplicação em funcionamento, com navegação real.

Tempo	Bloco	Conteúdo
0:00 a 0:25	Abertura	Nome, RM, turma e o problema: um canal de denúncias que pressupõe que alguém consiga digitar
0:25 a 0:55	Pitch	As três situações da seção 4.1: quem não pode digitar, quem não deveria precisar denunciar, quem não tem aparelho
0:55 a 1:20	Arquitetura	Um barramento, três origens. O aparelho relata, o servidor decide, o VigIA classifica pelo texto
1:20 a 1:50	Demonstração: pânico	Acionar duas vezes. Mostrar o repique reconhecido
1:50 a 2:35	Demonstração: tornozeleira	750 m, enviar. Sul, enviar. Sul, enviar: violação. Dizer em voz alta que a resposta não traz distância nem raio, e por quê
2:35 a 3:00	Demonstração: totem	TT-001 aceito, TT-003 recusado. Dizer que o totem não informa a própria posição
3:00 a 3:25	Painel: dispositivos	Contadores subindo e a coluna do que o servidor decidiu
3:25 a 4:05	Painel: focos	Os sete focos, e os dois marcados como fluxo contínuo. Explicar a diferença entre surto e volume normal
4:05 a 4:30	SQL	Mostrar a consulta 3, a que audita a janela de 120 segundos com `LAG`
4:30 a 4:40	Fechamento	136 testes, o que ficou de fora e o próximo passo

Preparação antes de gravar. Recarregar as páginas com Ctrl+Shift+R, para não demonstrar uma versão em cache. Subir a aplicação e o servidor HTTP com antecedência. Deixar o simulador e o painel em janelas lado a lado.